Challenge BinaryGecko

Eugenio D. Deluca

(KarasuRØØT) – Date: 08/10/2025

Index

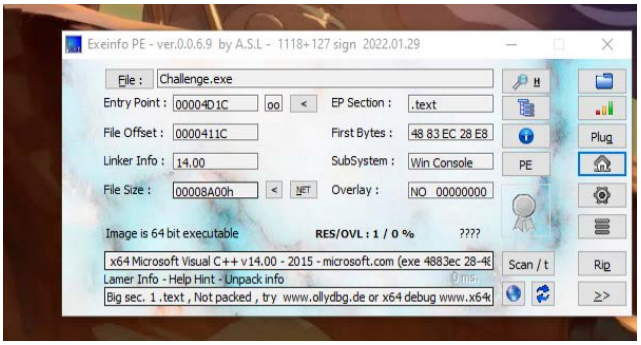
Level 0 (Intro)

This document details the methodology and findings obtained during the resolution of the Reverse Engineering Challenge BinaryGecko '.

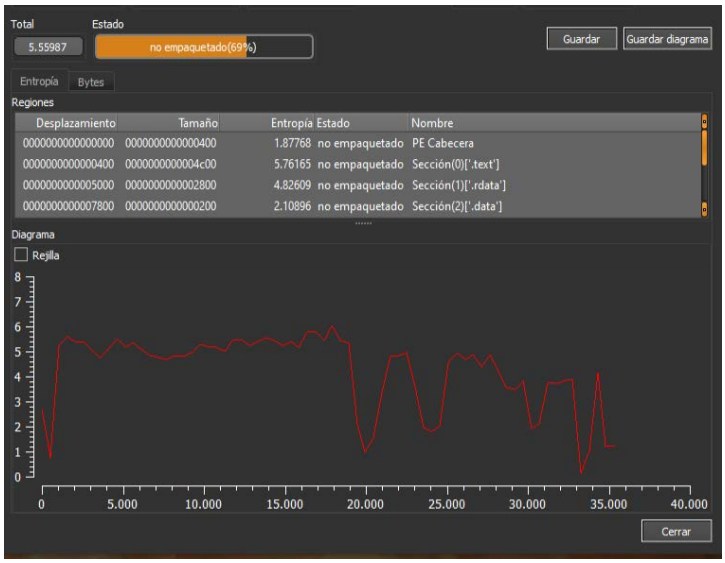
Tools:

- IDA PRO
- .x64
- Wit

As the instructions indicated, I changed the extension from “Challenge.txt” to “Challenge.exe” – Then, out of curiosity, I opened Exeinfo PE – to see what I was up against.



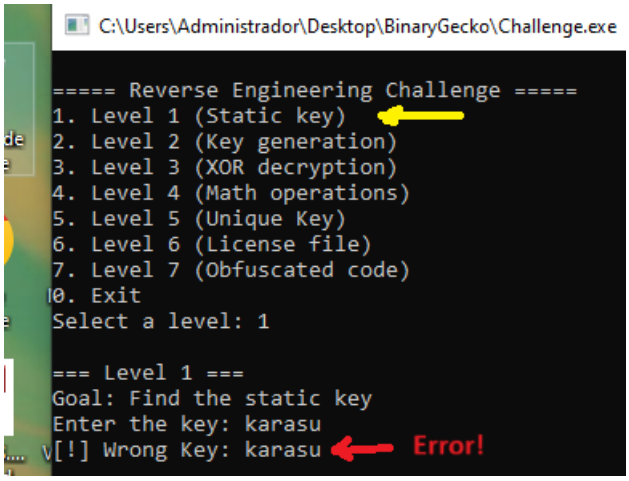
As you can see in that image, it is a 64-bit executable developed in C/C++ (bad news for me since I don't have much experience with 64-bit executables – but it's not a problem).



And very good news for me is not packaged this time.

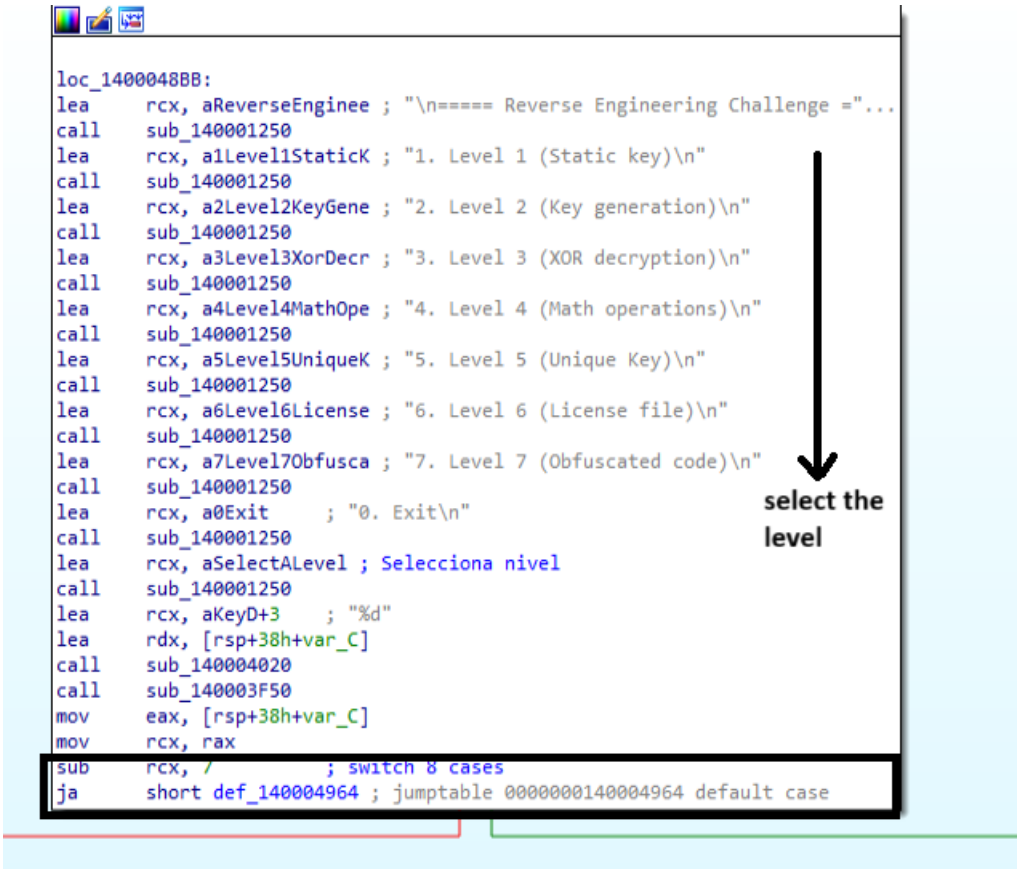
Level 1 (Static Key)

Well after this little “ intro ” we start with Level 1 (Sratic key)

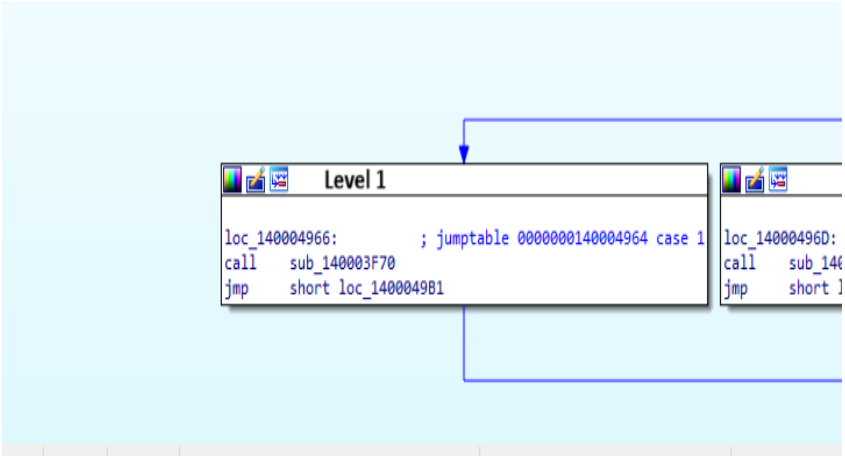


To do a quick test, point to a key as a test (“ karasu ”) and of course, it indicates that you are wrong.

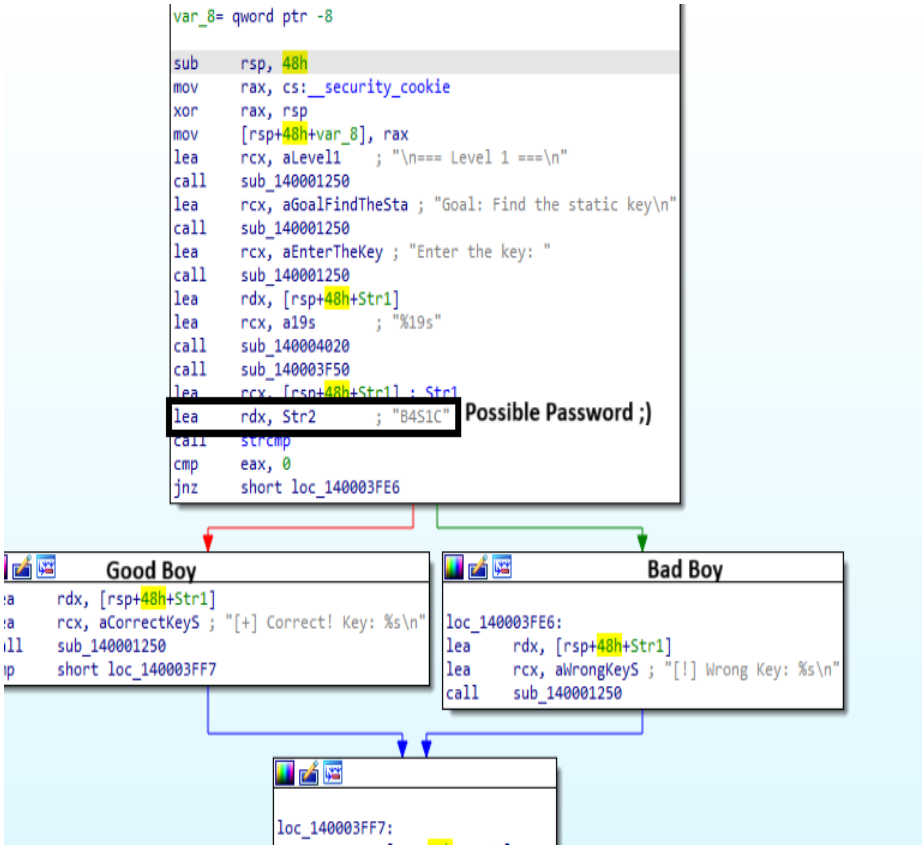
I start by opening IDA PRO:



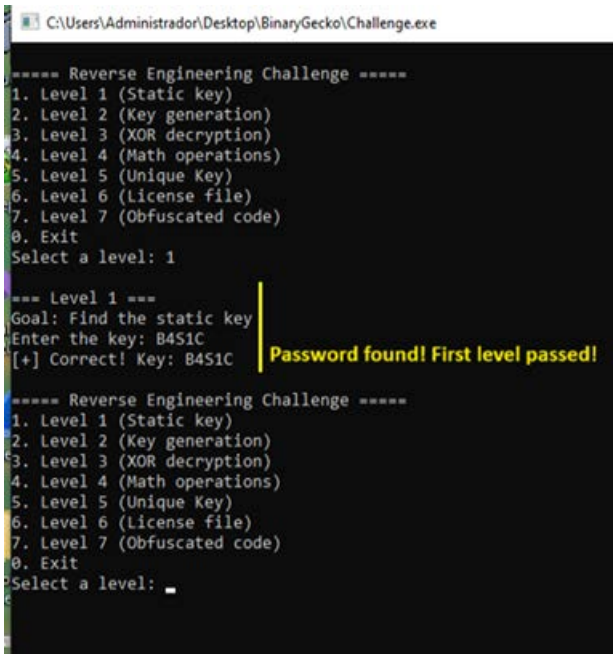
Of course, this analysis will be performed for the first and only time now. This means that, as we know, it's a program with a "case" statement, which will jump depending on the number we specify.



As we can see, this is the case for all seven difficulty levels of this challenge. For each level, there is a case and then a call to the corresponding function. Therefore, for this first level, I enter " call sub_140003F70."

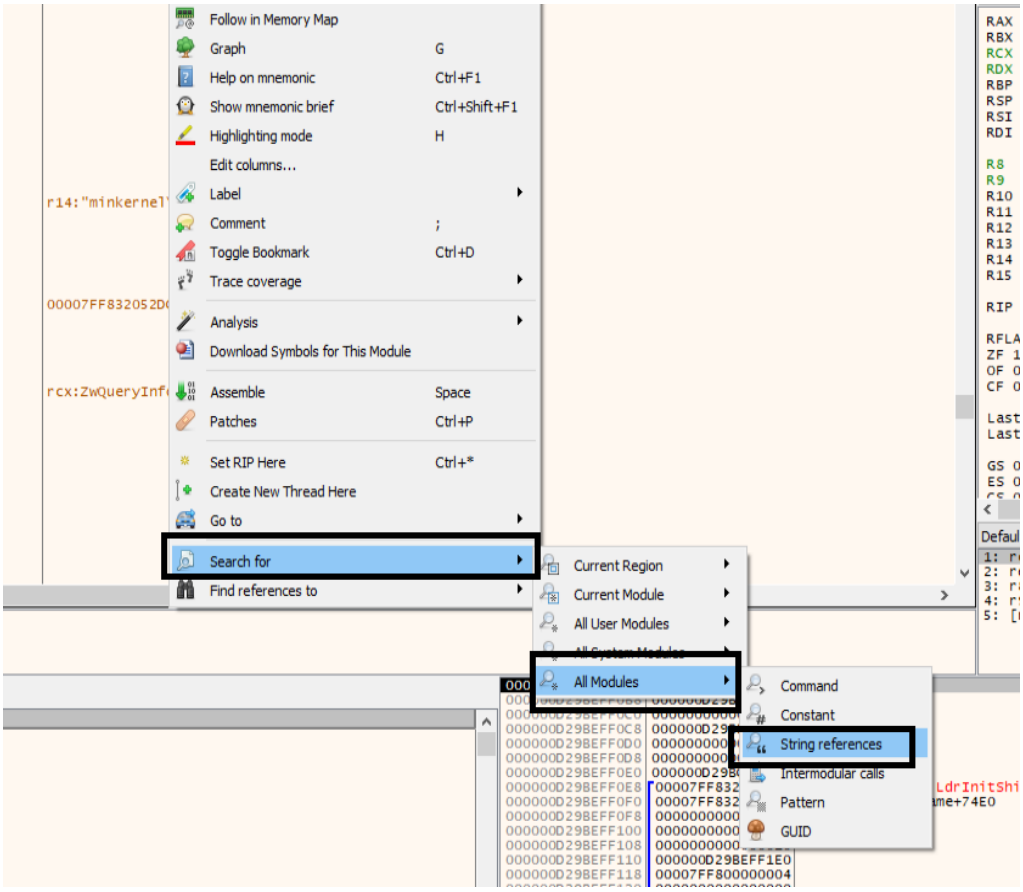


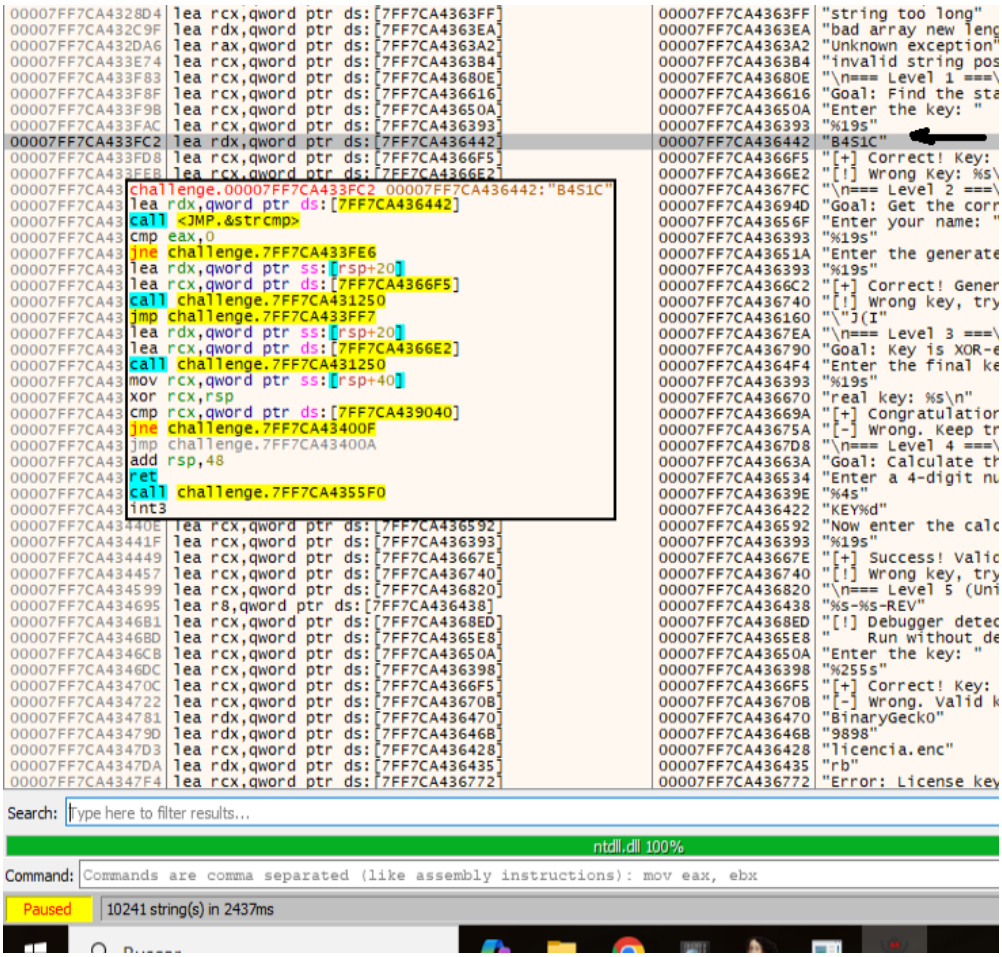
At this first level, we can clearly differentiate “Good Boy” and “Bad Boy” and additionally, the possible password “ B4S1C ” is hardcoded .



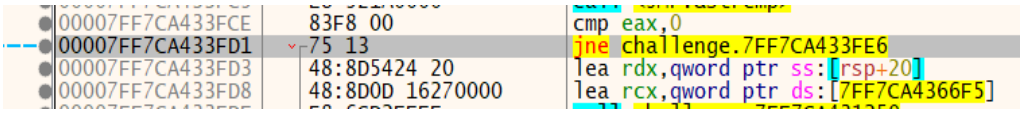
Perfect, the first level was successfully completed – So, just by checking the program flow in IDA PRO, we found it without any problems.

As an extra, it is also found on x64, simply by searching for the strings – right click -> Search for -> All Modules -> Strings references -

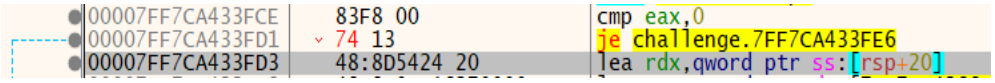




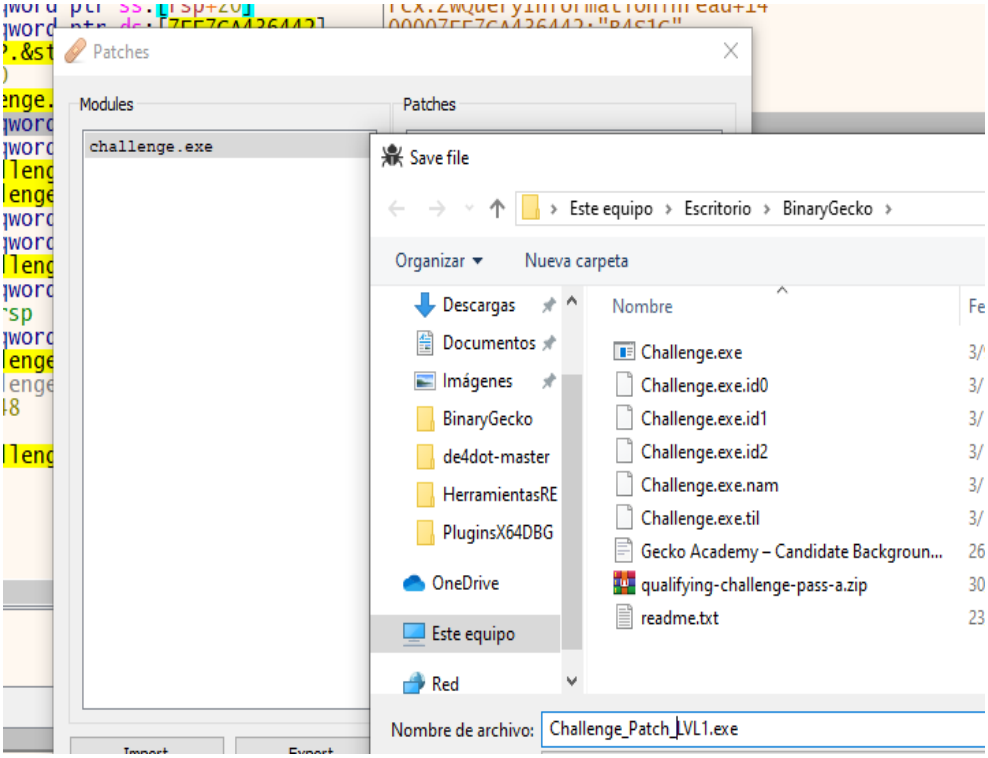
Since the password is hardcoded , the only option would be to patch it. The line to change is:



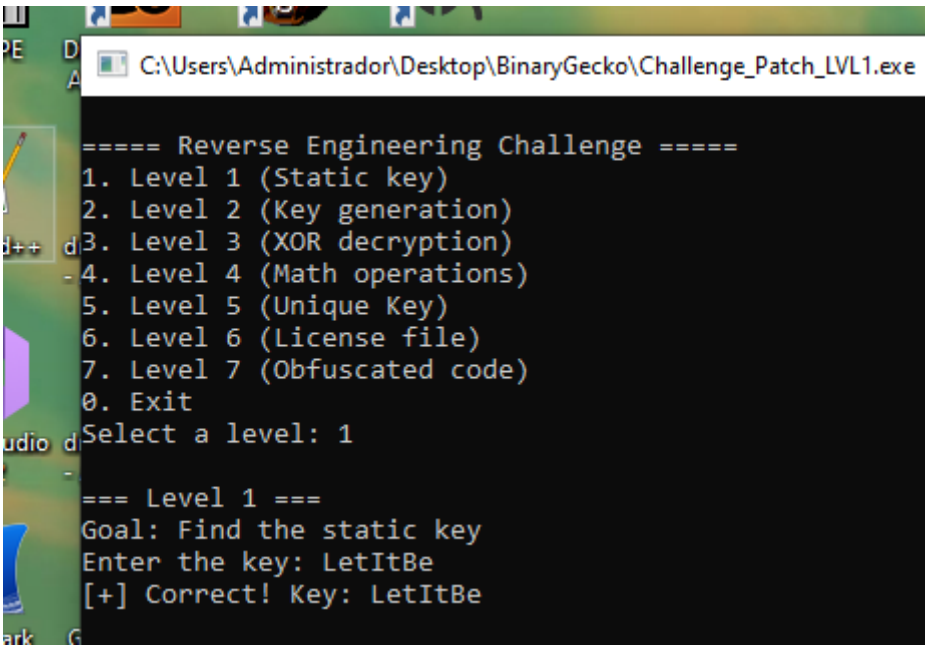
I decided to replace it with:



Then I patch the program:

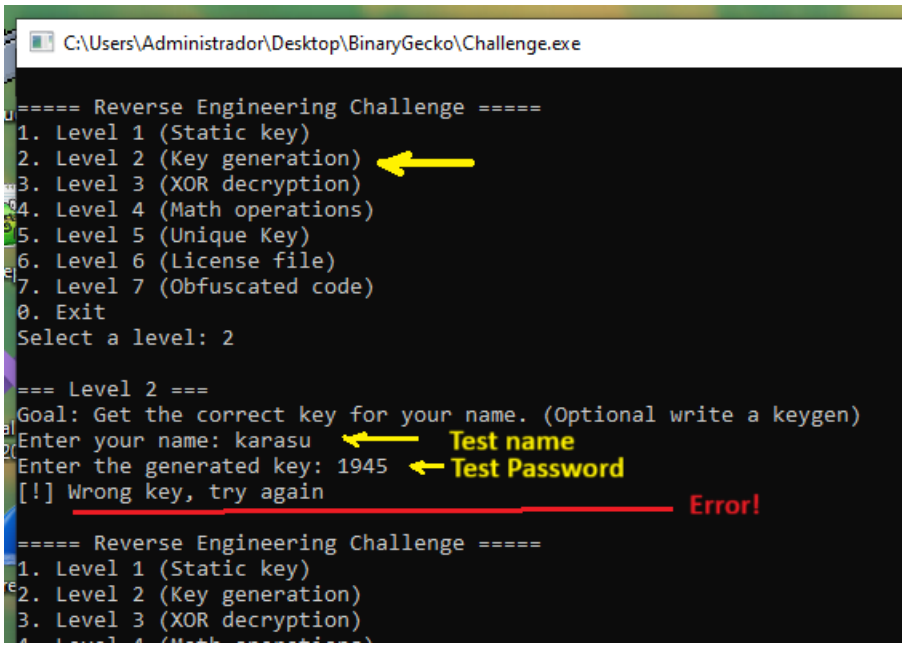


And I run it – I use any password – for example LetItBe The file “Challenge_Patch_LVL1.exe” will also be attached.



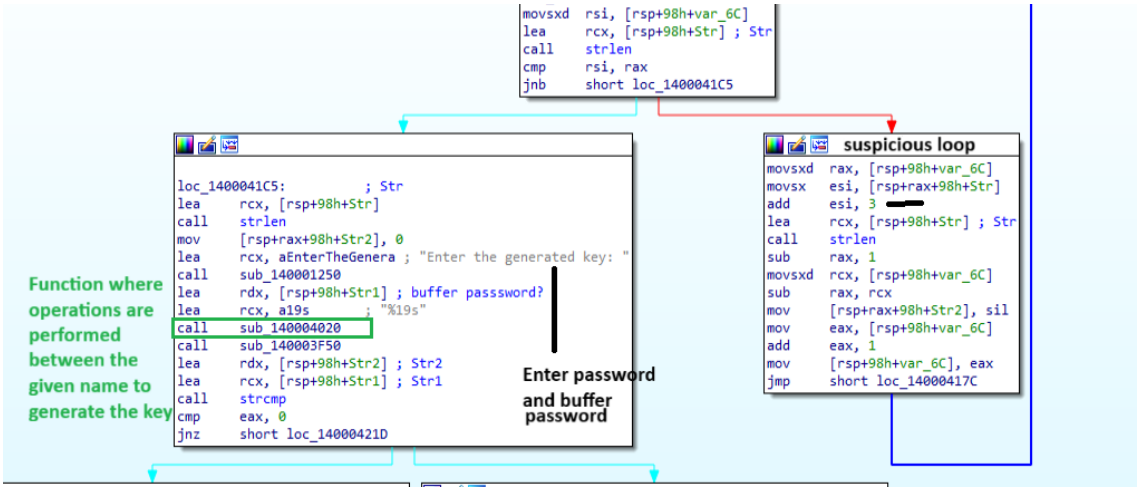
Level 2 (Key generation)

Let's start with Level 2:



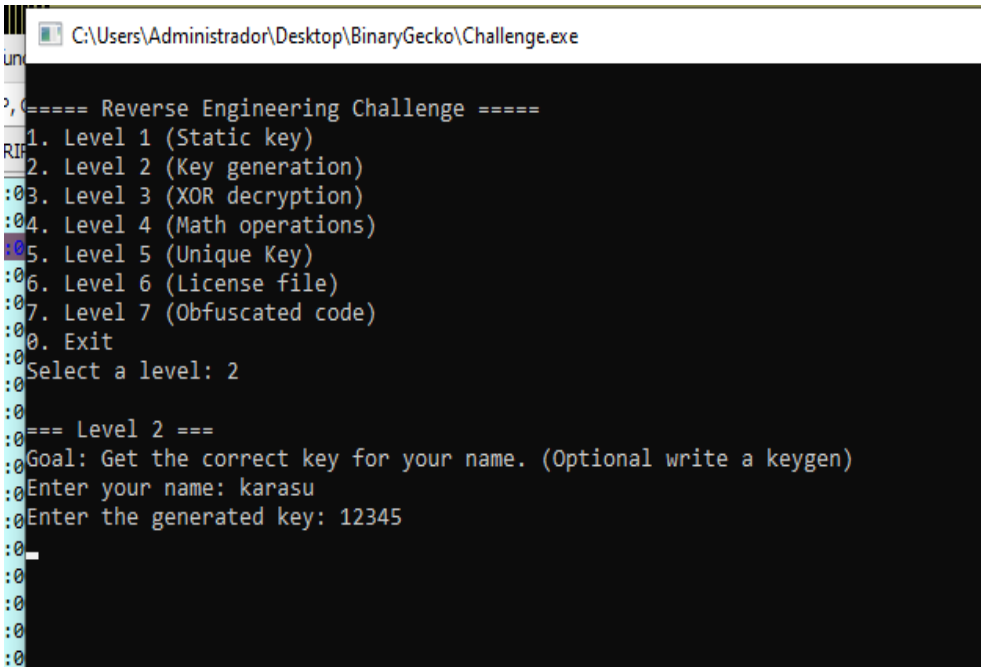
As we see in the previous image, there is where we enter the “ name ” and below is the “name buffer” – and in the image that follows, on the one hand, this is where we enter our key and its corresponding buffer, in addition to the Function where operations are performed between the given name to generate the key.

And then there's a loop which, as we see, for example, performs operations, so I assume it's responsible for performing operations with our user.

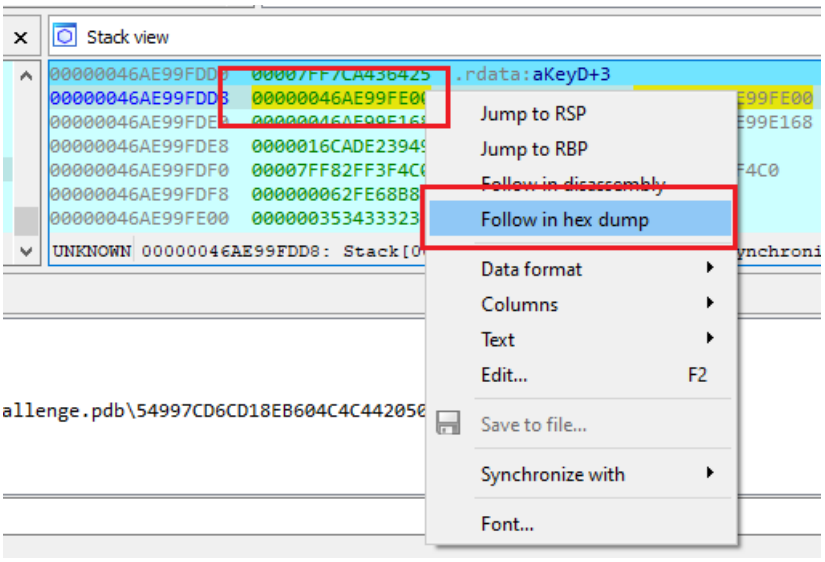


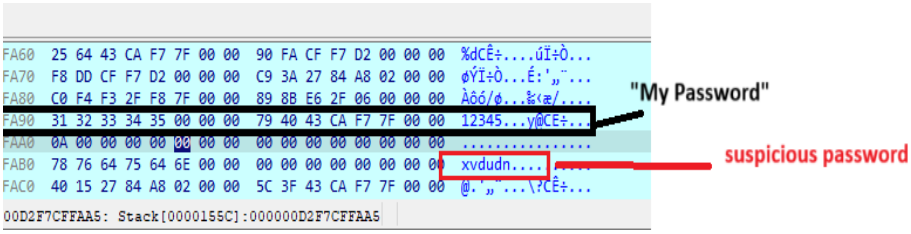
In order not to waste time, I just place a Breakpoint on the “lea rcx , [rsp+98h+Str1] ; Str1”.

As I did in the previous exercise, I enter a test name (“ karasu ”) and password (“12345”).

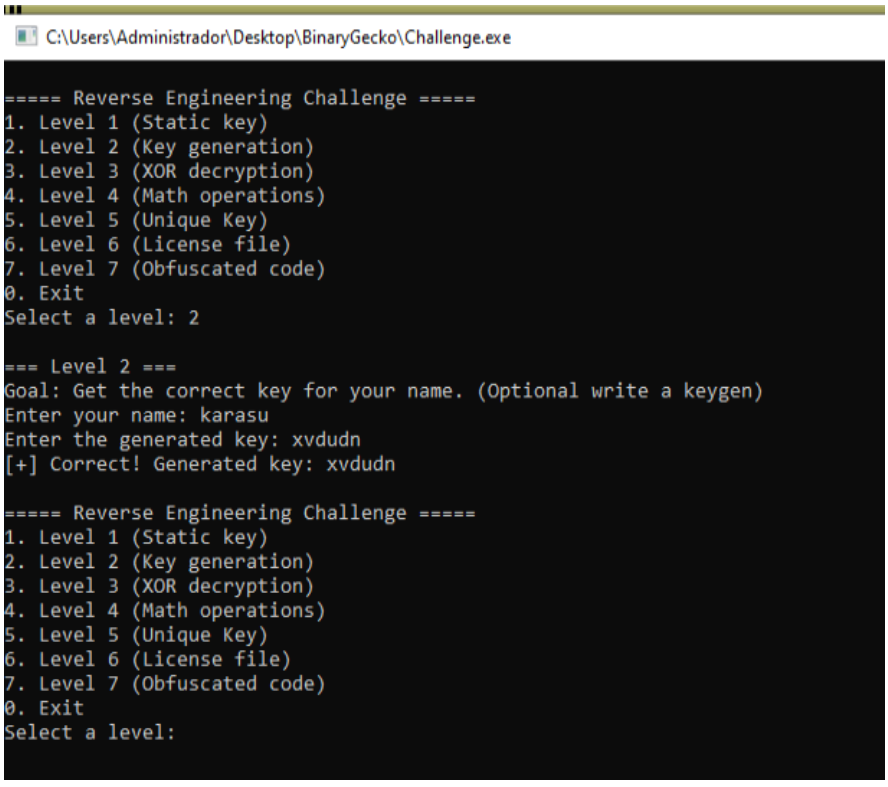


There the program stops, so I thought I'd check the Stack – Follow in hex dump – on the line “00000046AE99FDD8 00000046AE99FE00”.

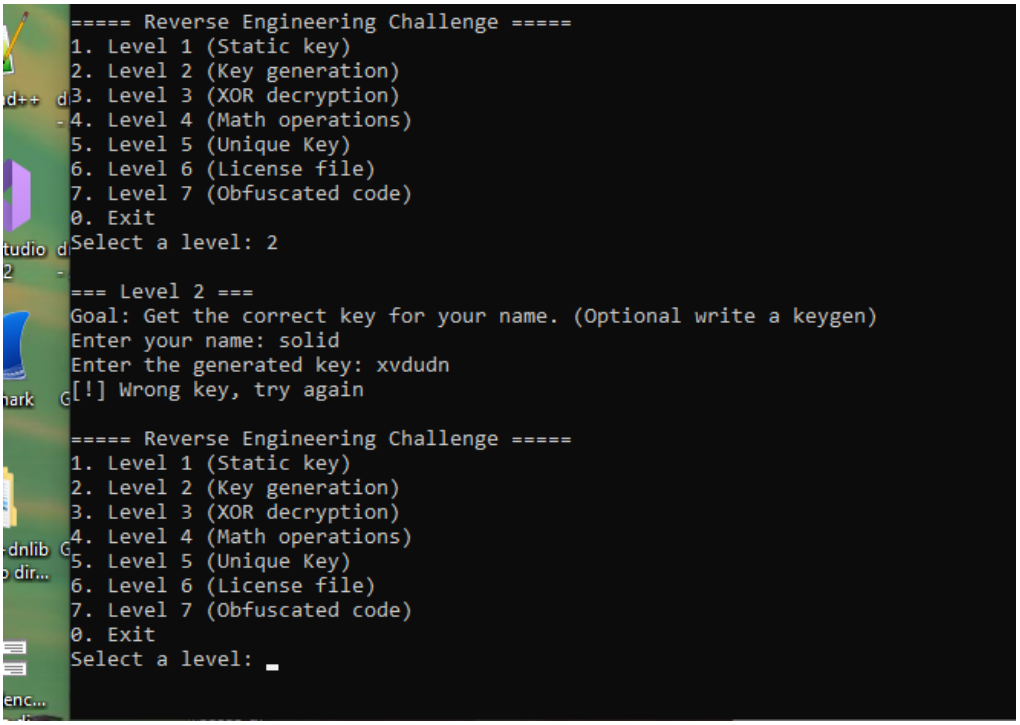




Then the program continues and ends up indicating that it is not the correct password, so I decided to try brute force style – then, name (“ karasu ”) and password (“ xvdudn ”) and it is accepted correctly.

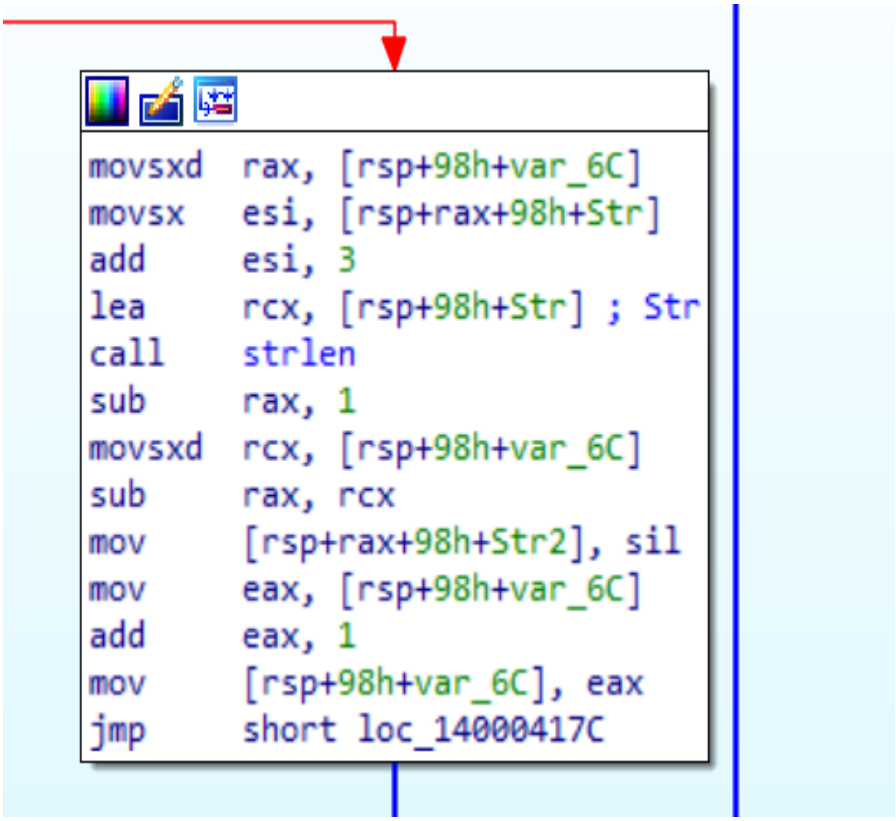


Yes , it does, but wait, someone once said to me: "Why don't you try another name?" So I tried it:



Of course it didn't work.

After analyzing it carefully, I realized that the answer was right there, and it was the famous loop.

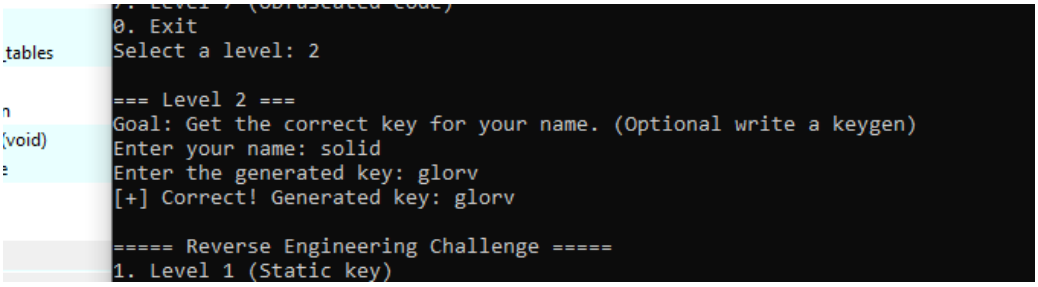


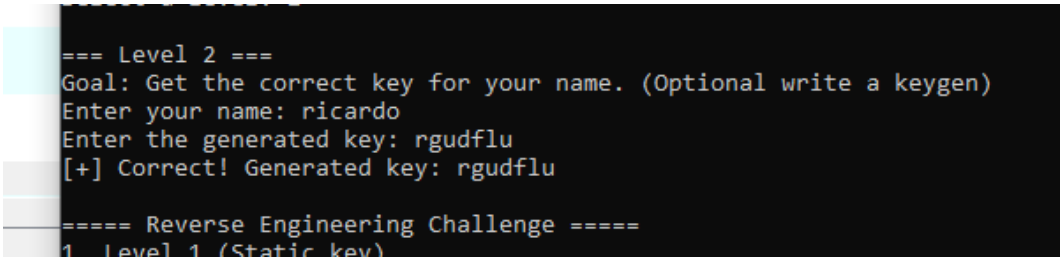
Without a doubt, what the loop does is take the previously typed user string and go through it, adding +3 to each character – then inverting it. Example with karasu →

Input Character	Original ASCII Value	Operation (ASCII+3)	New ASCII Value	Cipher Character
k	107	107+3	110	n
to	97	97+3	100	d
r	114	114+3	117	or
to	97	97+3	100	d
s	115	115+3	118	v
or	117	117+3	120	x

At first, I must admit, I fell into the trap. Wanting to speed things up, I tried the username karasu and the password ndudvx , and logically, I failed. But after a few minutes, I realized I was missing a step: inverting it. That is, the program itself takes each character and then inverts it.

Perform some tests for example:

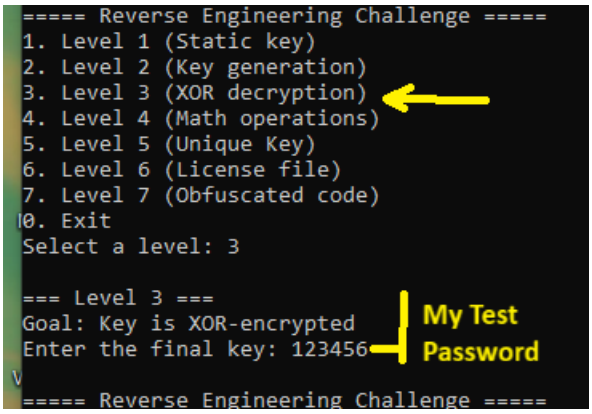




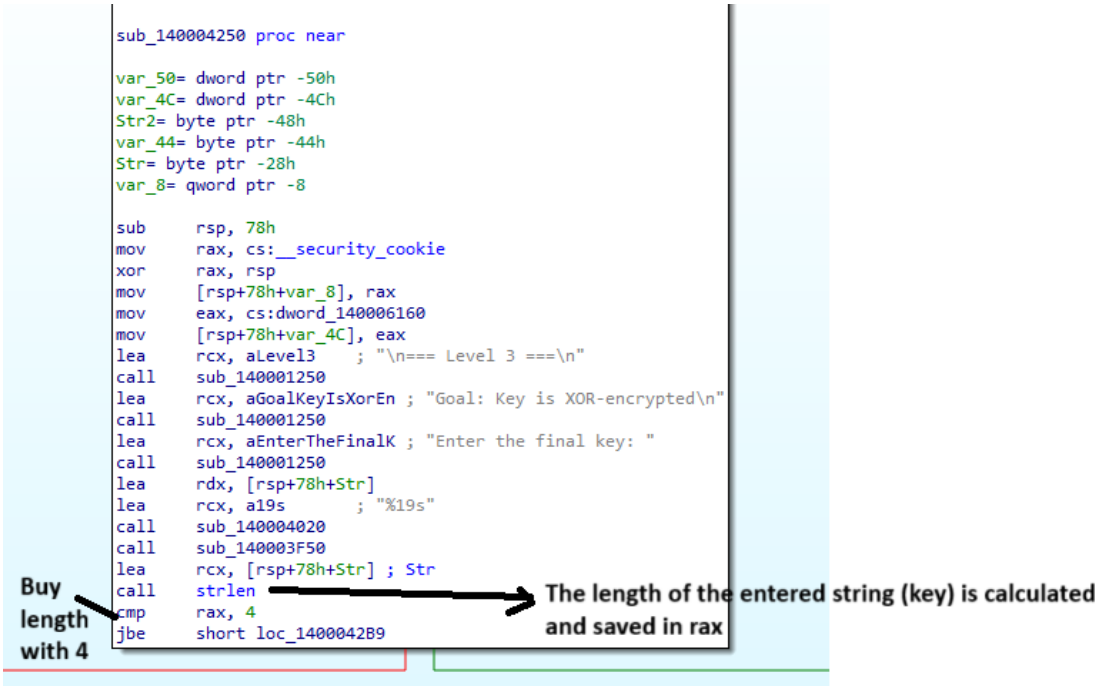
So, my theory was correct. I designed the script, a screenshot of which I'll include here, and I'll attach it as "Challenge2-BinaryGeckoENG.py."

Level 3 (XOR decryption)

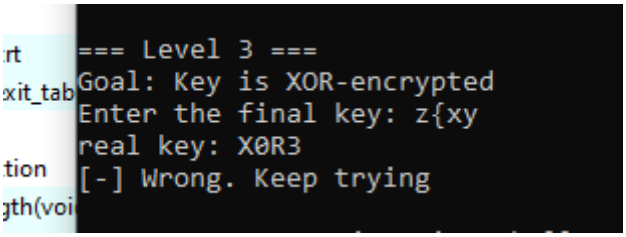
Let's start with level 3:



Well, the first problem I had was that I gave it any password, but it didn't tell me anything else, so I opened IDA and started trying to analyze the situation.

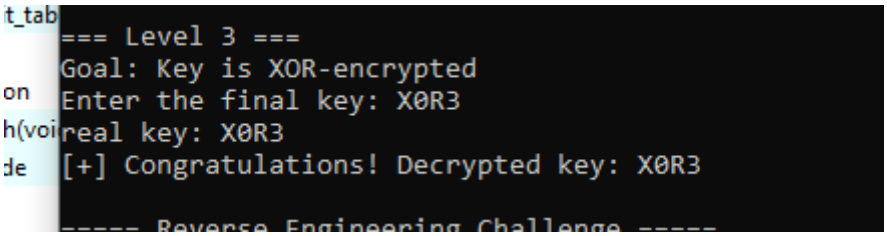


Since I'm just starting out with reverse engineering, there are things I need to review and double-check. And in one of the tests, it appeared, and seeing the previous screen, of course, my first test password didn't work because I tried 6 characters (123456) and this time I tried 4 random ones (z{ xy }):

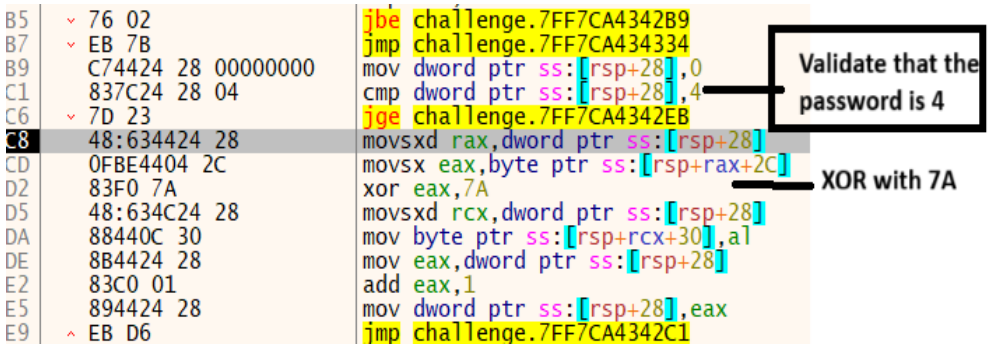


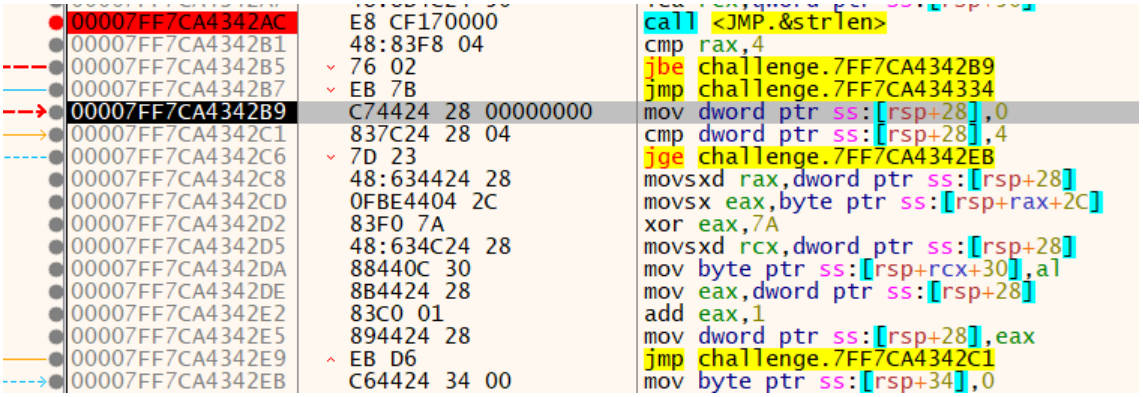
Is the X0R3 key the real one? There's no one good enough in this world to tell you the correct key. Forgive me if I doubt these intentions, but I decided to give it a try.

And indeed, in theory that is the key:

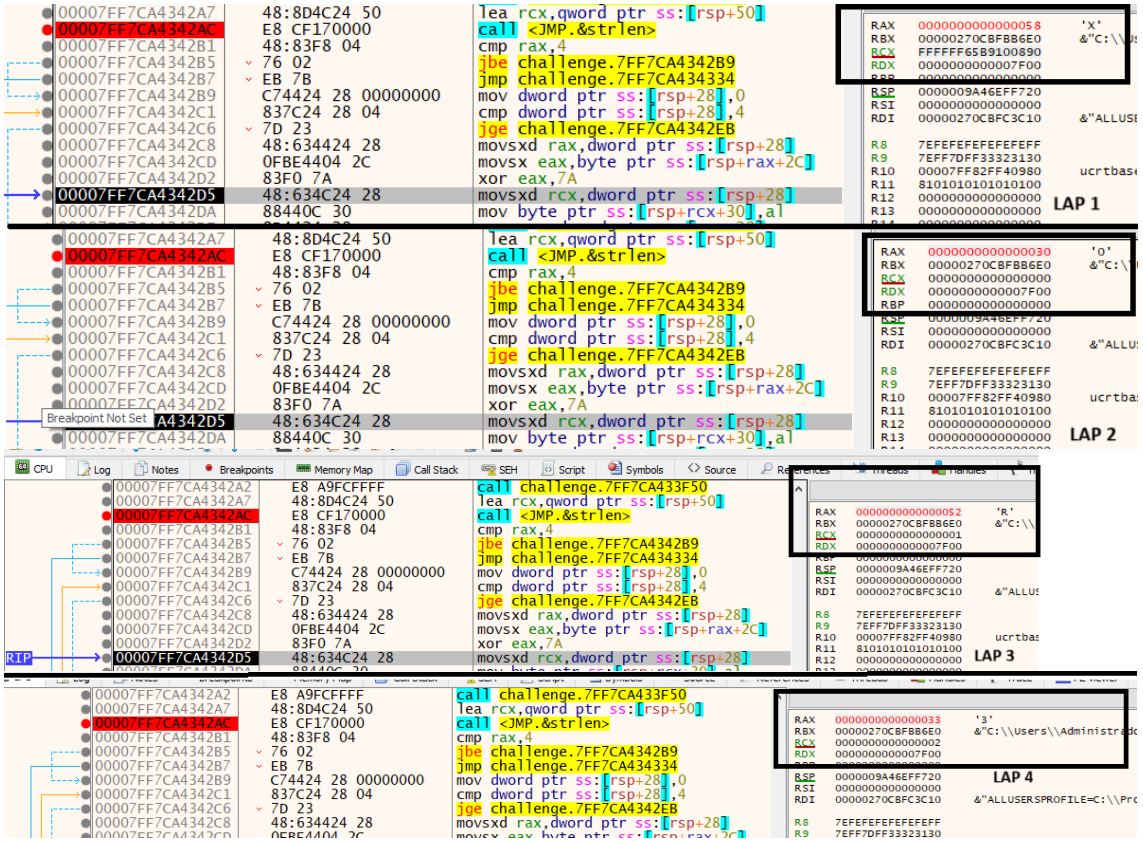


But since it's a BinaryGecko challenge and I'm already at difficulty level 3, I don't believe it's that easy, so I did some more research. I'll use x64 for dynamic review.





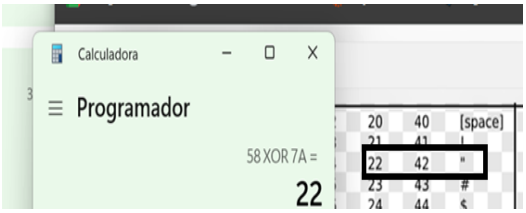
What happens here is that it enters the loop, compares if it doesn't reach 4 and continues, if it exceeds 4 it exits the loop.

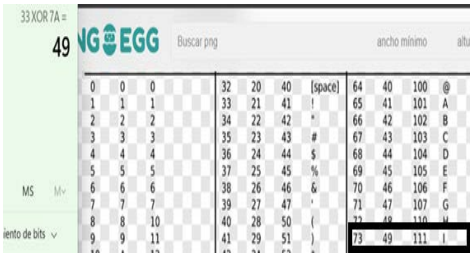


And in these screenshots you can see how with each passing lap, it generates the X0R key. Then I decided to review this exercise again. I really thought that it couldn't be the only answer, so I started to investigate a little more.

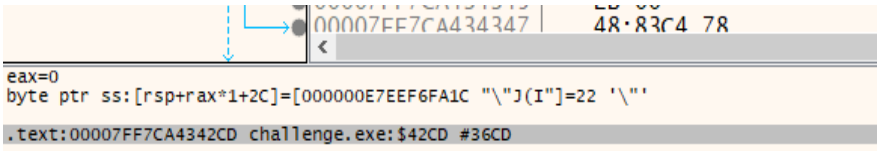
My hypothesis is that X0R3 is the “decrypted” key, so to speak. So perhaps finding the correct key.

What the program itself does is perform the XOR EAX ,7A operation , so this means that it will do the following:

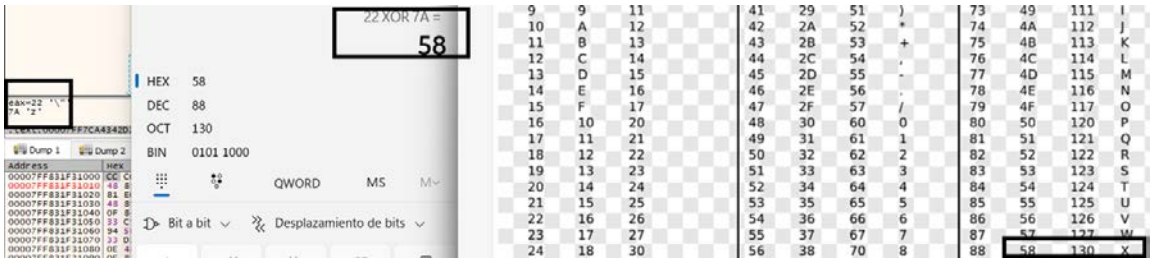


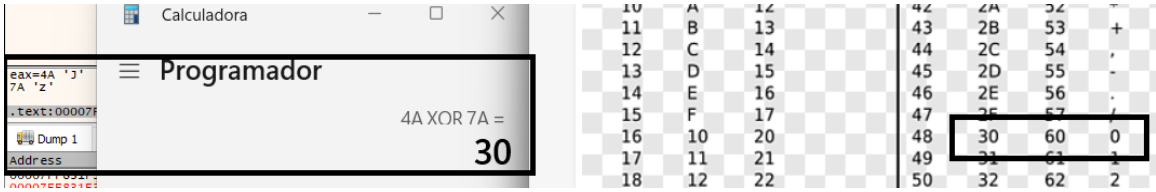


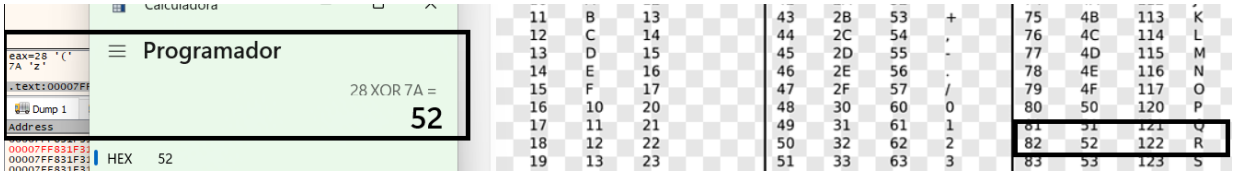
Additionally to justify it, the x64 indicates:

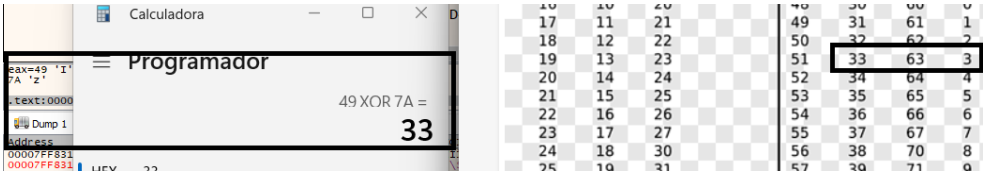


If we follow the instructions:

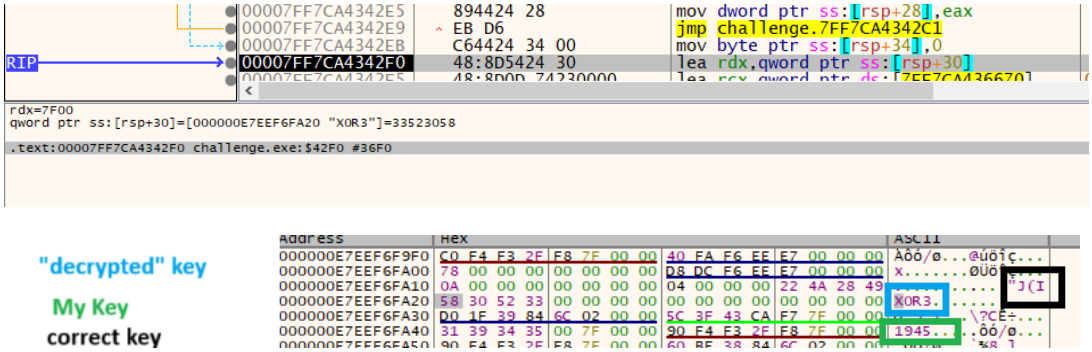




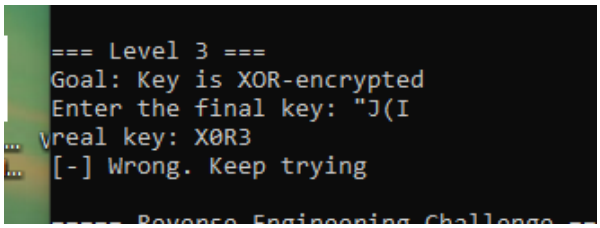


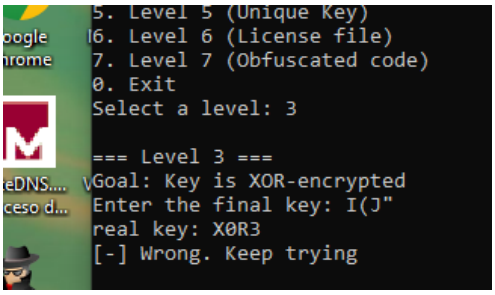


So then:



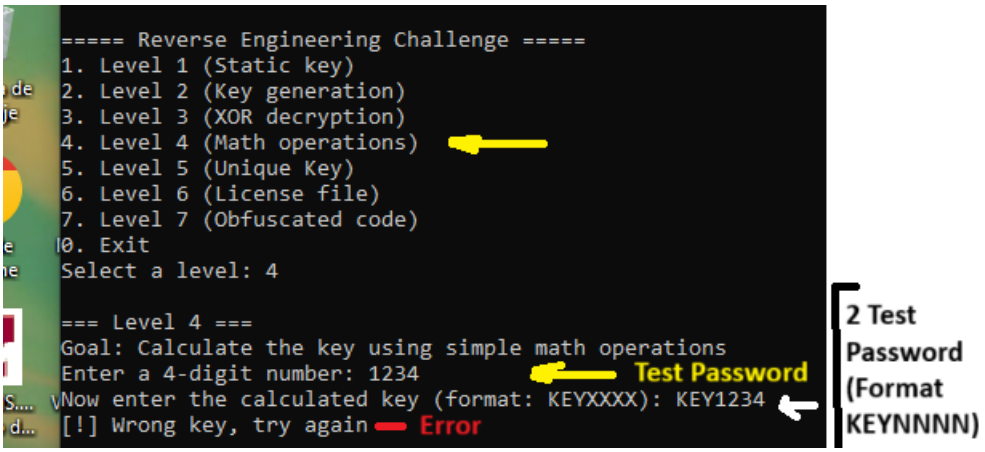
So the result should be **“ J(I** – or if it were the inverse – **I(J”**





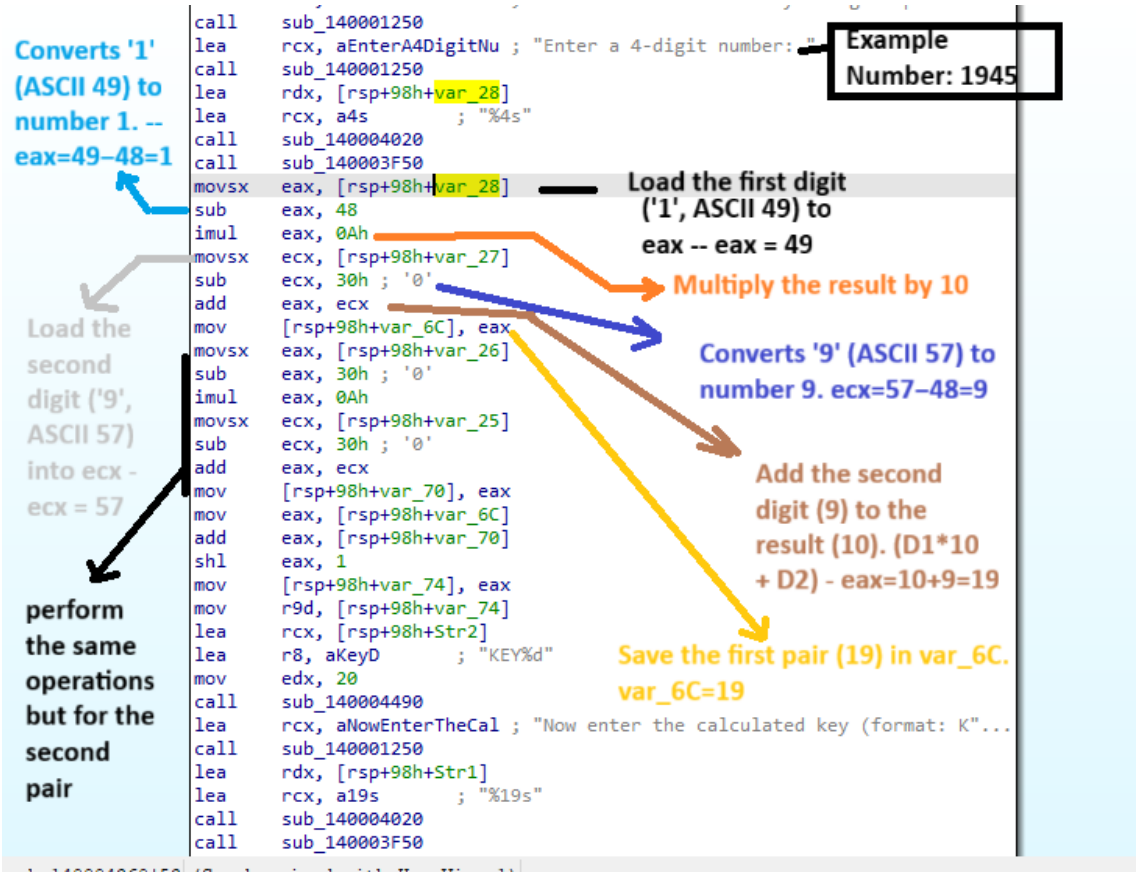
In short, the only accepted key is obviously X0R3 –

Level 4 (Math Operations)



Okay, let's start with the fourth challenge. First, it asks the user to enter four numbers. Then, it asks again for the calculated key in the format "KEYXXXX" -

After analyzing it statically and doing some dynamic testing, I realized it wasn't as problematic as it seemed. It just performs several operations, and that sometimes causes you to get lost.

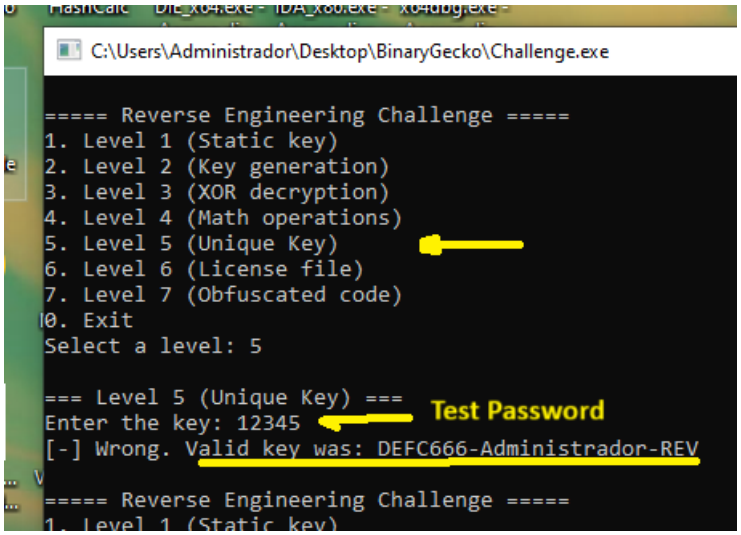


I apologize in advance for so many colors, but I couldn't think of any other way to explain it better.

But in short what the program does is – Requests a 4 digit number (in our example it will be 1945) – then separates it in two – that is, takes 19 on one side – and 45 on the other – then adds them →19 + 45 = 64 – then multiplies it by 2 – 64 x 2 = 128 and then concatenates the word KEY – Logically, then compares “KEY128” with the 2nd key that is entered and if they are not equal, it jumps to “bad guy” and ends. If not, it jumps to “good guy” and we solved the exercise.

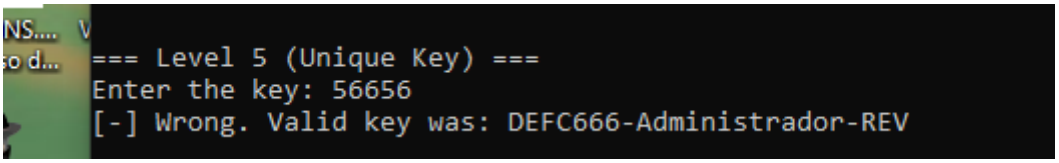
Although not expressly requested, I took the liberty of making a small keygen – I will attach said script with the name “Challenge4-BinaryGeckoENG.py” referring to this point.

Level 5 (Unique Key)

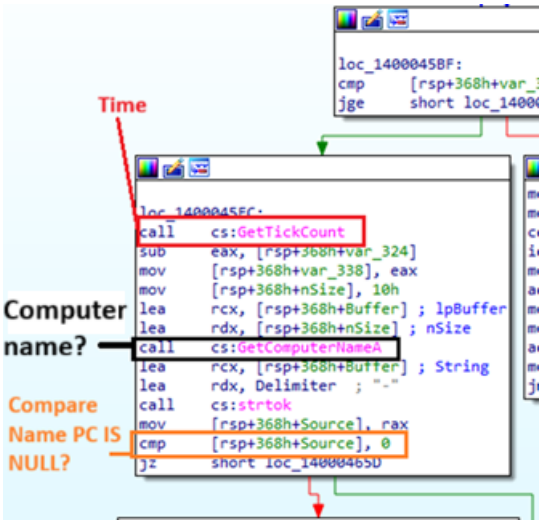


Well, starting challenge number 5, they ask me for a password (it doesn't specify the format or length. To my surprise, it not only reports that that is not the password and that the valid password is "DEFC666-Administrator-REV".

I tried a completely different key, but it reports exactly the same thing:



When I start it in IDA PRO – and do a quick check, I notice the following:

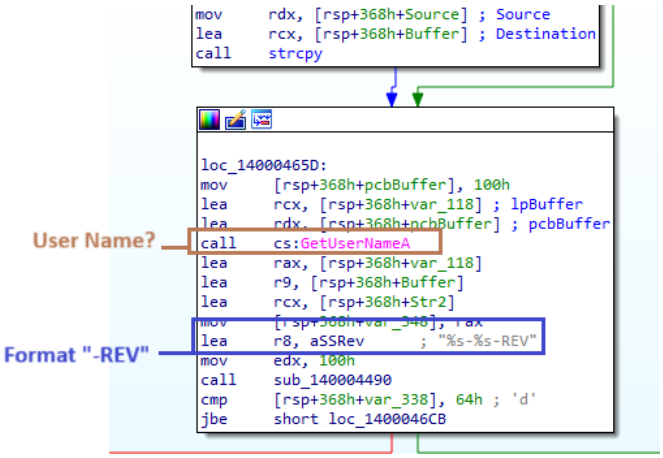


Well, it seems we have some details here that I would like to express.

According to MSDN “ GetTickCount ”: Retrieves the number of milliseconds that have elapsed since the system started, up to 49.7 days.

According to MSDN, “ GetComputerNameA ”: Retrieves the NetBIOS name of the local computer. This name is set at system startup, when the system reads it from the registry.

also compares/checks (at least it's my estimate based on how it is given [RSP+368h+Source], 0 – if a token was found (if the team name was not empty).

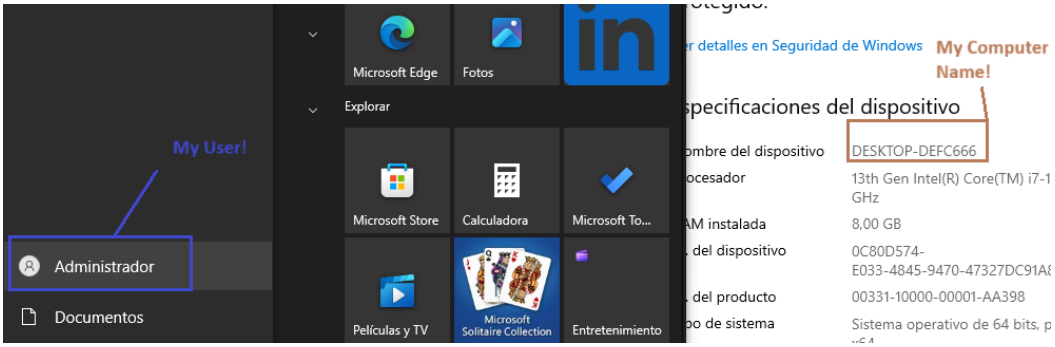


In the image above, it shows us two important things:

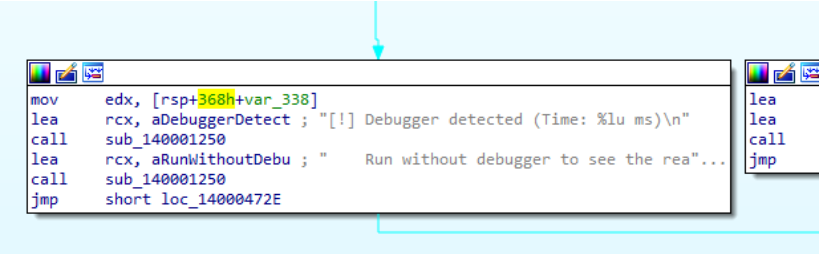
According to MSDN “ GetUserNameA ”: Retrieves the name of the user associated with the current thread.

Also, below prepare to have a -REV format

If we think about it for a minute it makes some sense if we compare it with what the program told us when we tried – it told us that the key was “DEFC666-Administrator-REV” – So it makes sense, since it takes the name of the PC + The name of the user and concatenates “-REV” In my environment:



Of course, this also has “Good Guy” and “Bad Guy” but it also has a block of code for Antidebugger that is related to time.

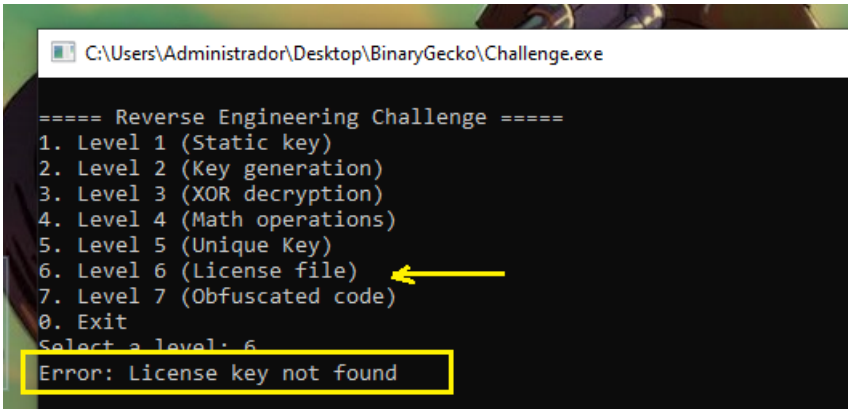


Now, since I also need to practice some programming, it occurred to me to make another Script (it's not really necessary, but I'll do it anyway).

It will be called Challenge5-BinaryGeckoENG.py and will be shipped along with this document and the other scripts.

Level 6 (License file)

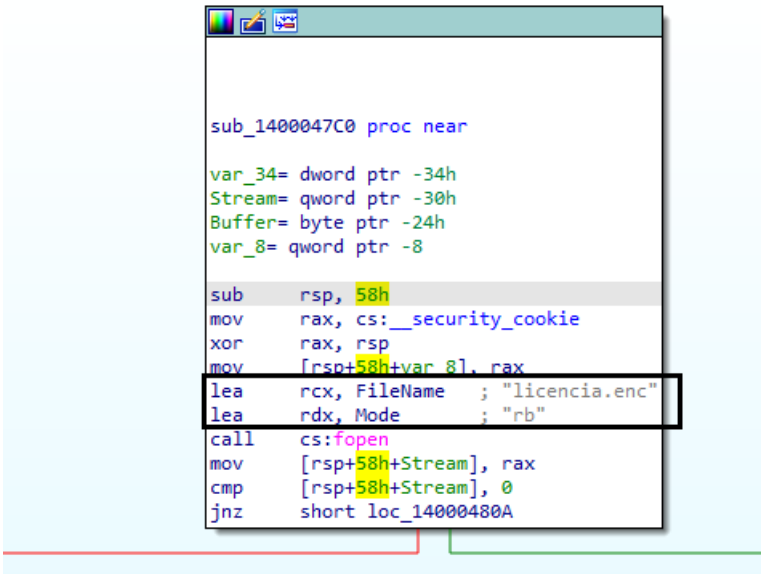
Let's start the penultimate challenge, let's execute it first.



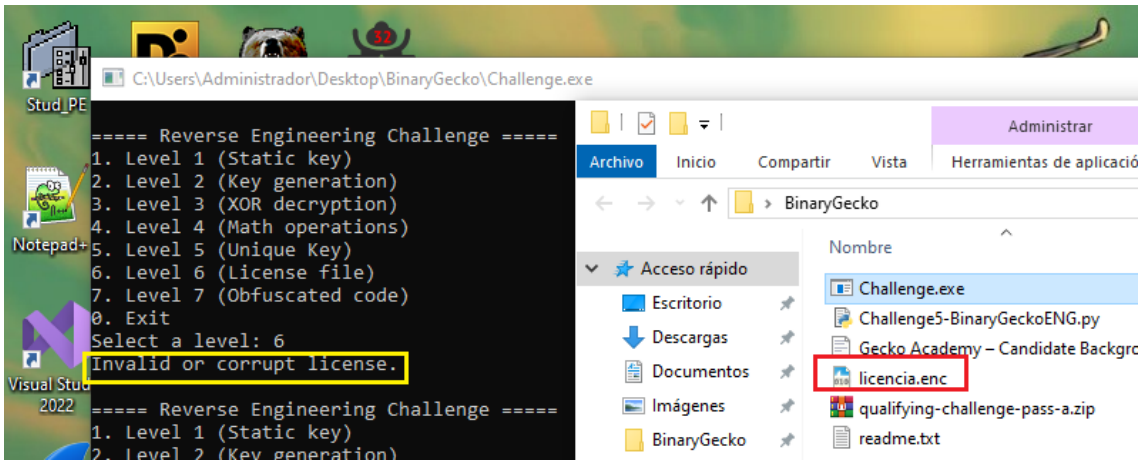
Ok, so as soon as I start it, it says “Error: License key not found ”

This means it is expecting a file, in my experience.

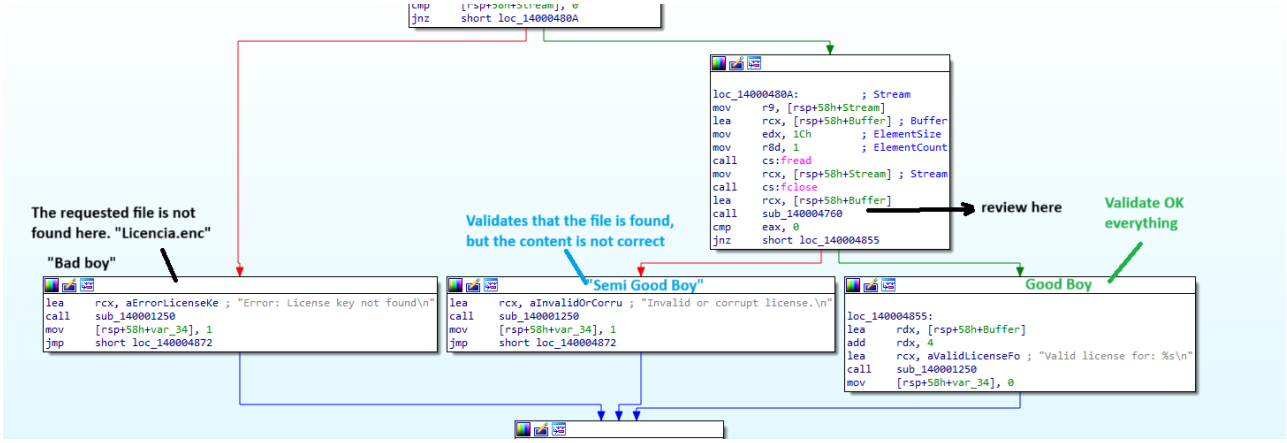
When opening in IDA Pro – it is very clear what I mentioned before



licencia.enc ” file , but it also has permission to at least read it.



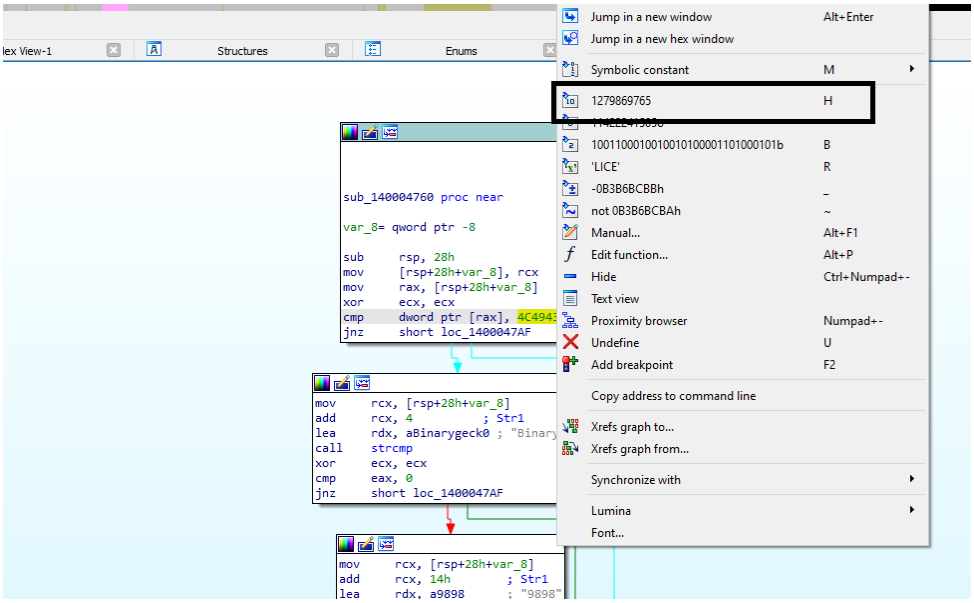
To validate my previous idea, I decided to create a file called " licencia.enc "—it's actually a notepad with that name—and I changed the extension. Now, we see that the message has changed, confirming both things: first, that it needs that file, and second, that it evaluates its contents.

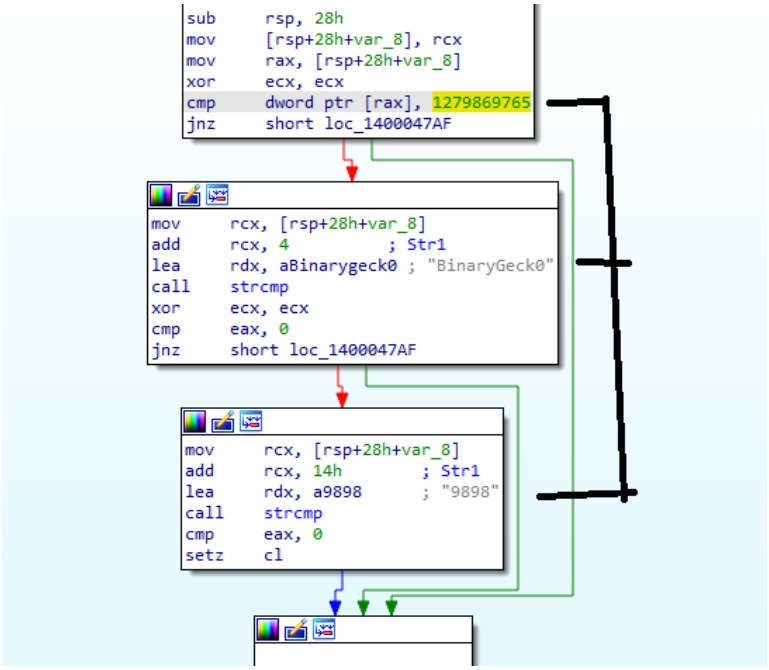


In this image, in case it is not clear, sorry I thought it was smaller – we have a function to investigate, then 3 paths which I defined as “bad boy” which is when the licencia.enc file does not exist – Then there is “semi-good boy” – which validates that the file is there but also that the content is correct – and “good boy” which validates that the file and its content are OK.

Then in the pending function to review I find that it compares with the constant “1279869765” and it is not a minor piece of data, since in a sequence of ASCII characters it corresponds to the string “Lice”

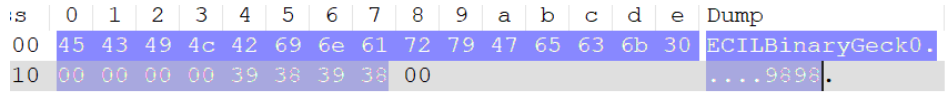
This is because: 1279869765 in Big Endian →4C,49,43,45 LICE // in Little Endian it would be ECIL (45 43 49 4C)





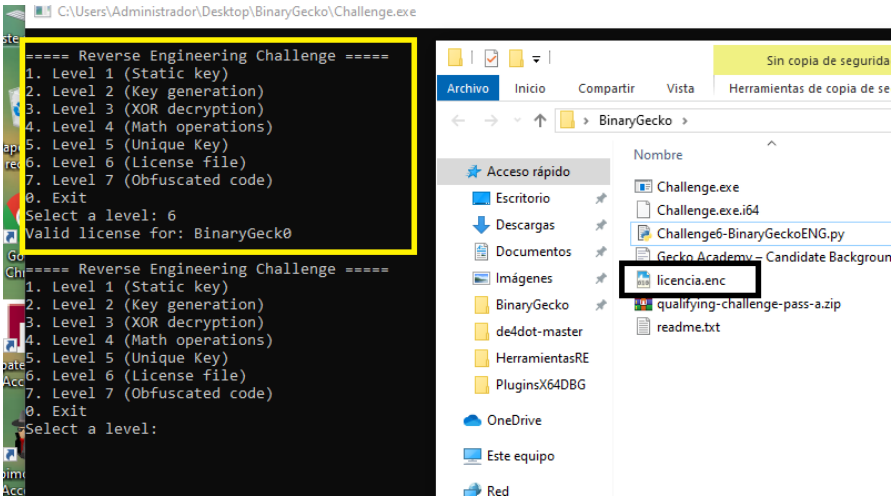
And then we see that there is the string BinaryGeck0 and 9898 – so, gathering all this information, it occurred to me that the file could contain the following strings “LICE” + “BinaryGeck0” + “9898” – I’ll be honest on this point too, the truth is that I tried several ways. And I always saved and tried different combinations and it never worked and I didn’t understand why, until I remembered a similar exercise, which instead of writing it manually in a document like a txt – I had to write it the same way in bytes.

I.e. , for example “ECIL+BinaryGeck0 +9898” – so I kept testing and it kept failing, until I realized that’s how I was writing it in the file. I finally realized that I should write it like this (I’m opening the Licencia.enc file with Notepad ++ and using the “HEX Editor” plugin)



-

I needed to complete the sequence with 0 or something else so that it would take the key correctly (I would sincerely like to clear this up at some point because I didn't really understand the reason - I imagine it's because it needs to complete the number of bytes).

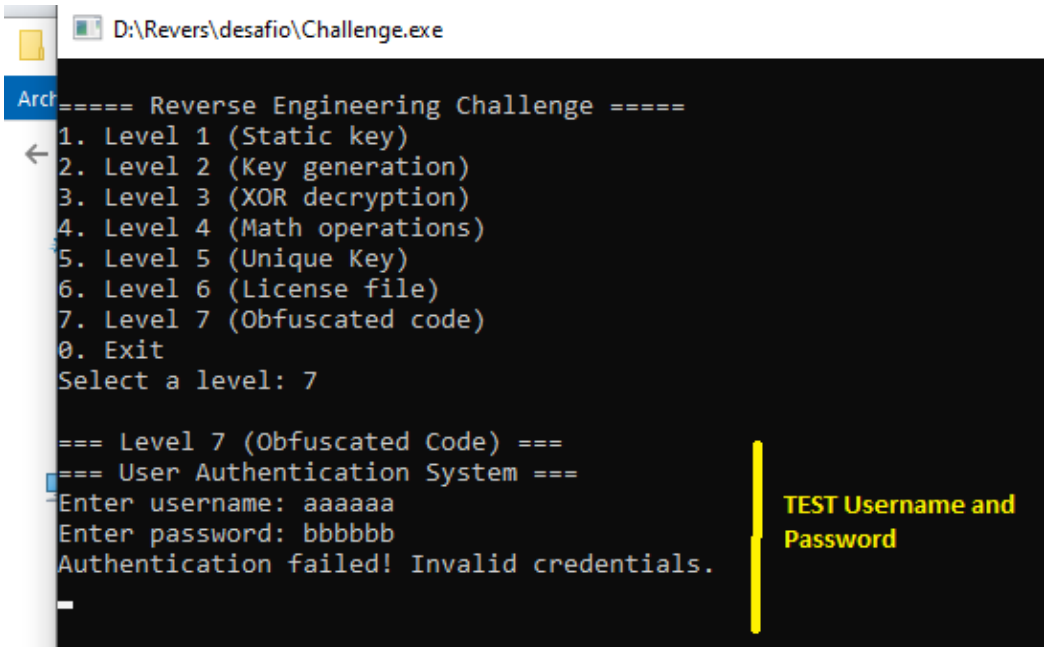


So, we can say that I finally found the solution, creating the file “licencia.enc” with the content already seen, when executing it it validates OK –

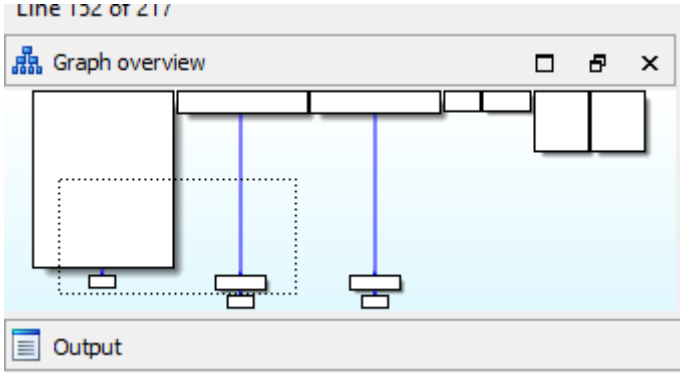
Of course, I generated a script (which, like the previous ones, will also be attached) with the name “Challenge6-BinaryGeckoENG.py”

Level 7 (Obsucated code)

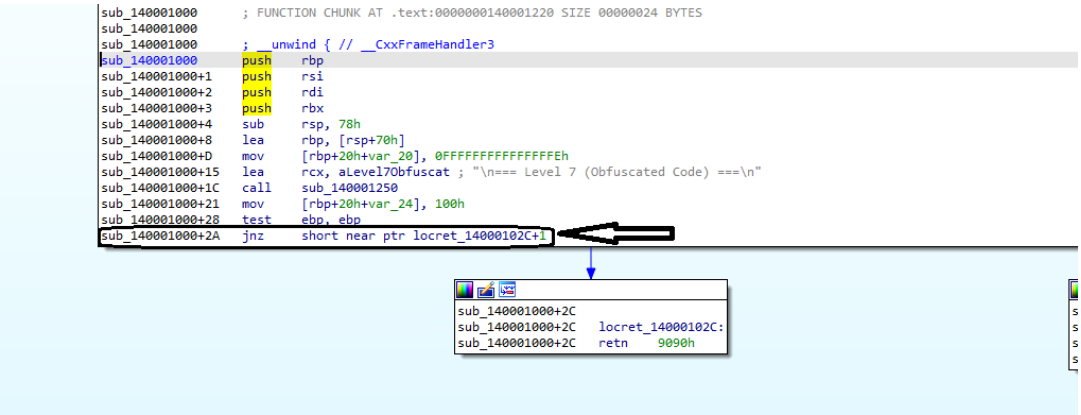
When starting the last challenge, it directly asks me for a username and password, and since it is not correct, it tells us “ Authentication failed ! Invalid credentails . And then the executable ends.



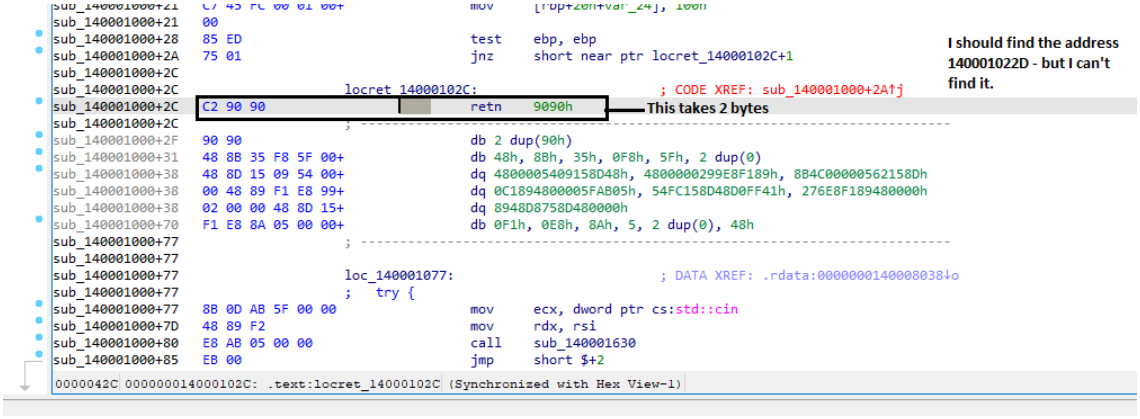
Indeed the code is obfuscated, we can notice it by some signs like this:



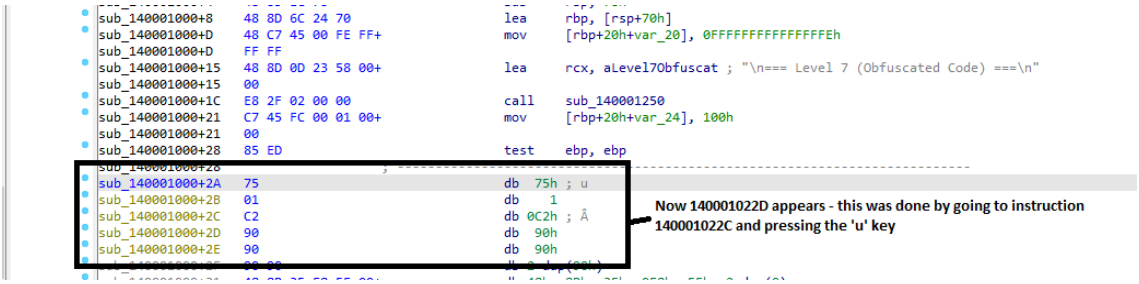
As proof, I open IDA Pro and see lines like the one marked – which indicate +”n” – in this case +1 – that is because it does not identify correctly.

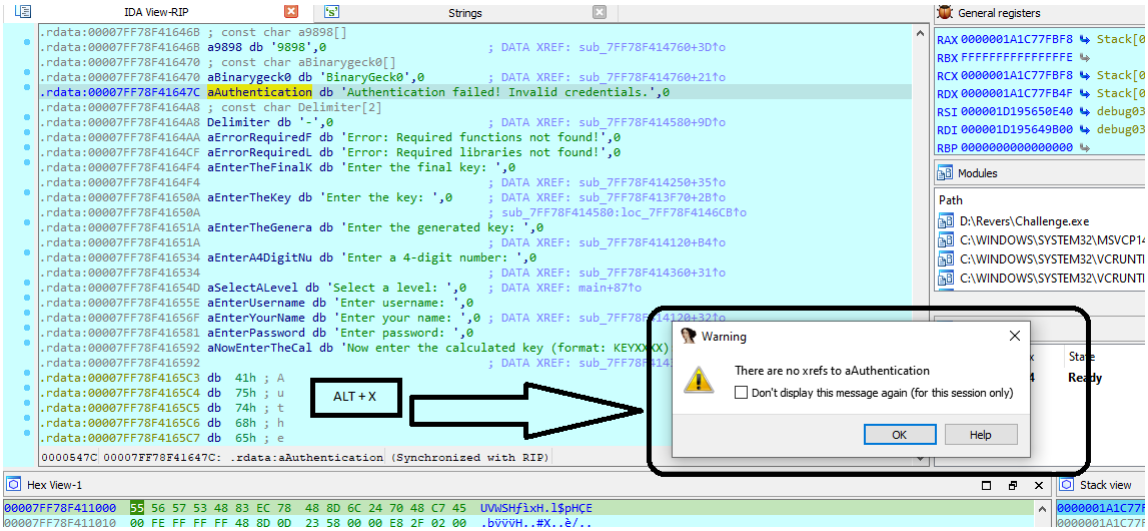


Here it clearly indicates that there is a C2 90 90 and in theory it should go to address 140001022D – but I can't find it here.



Then, we could go through the extensive code and rebuild everything that is allowed.



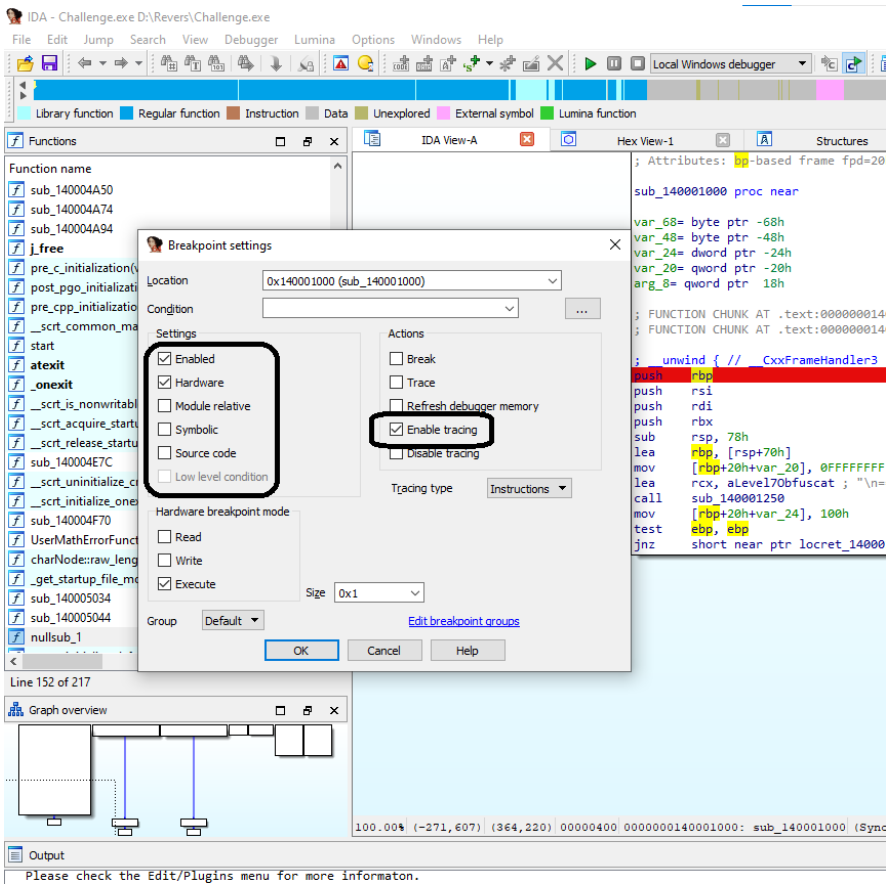


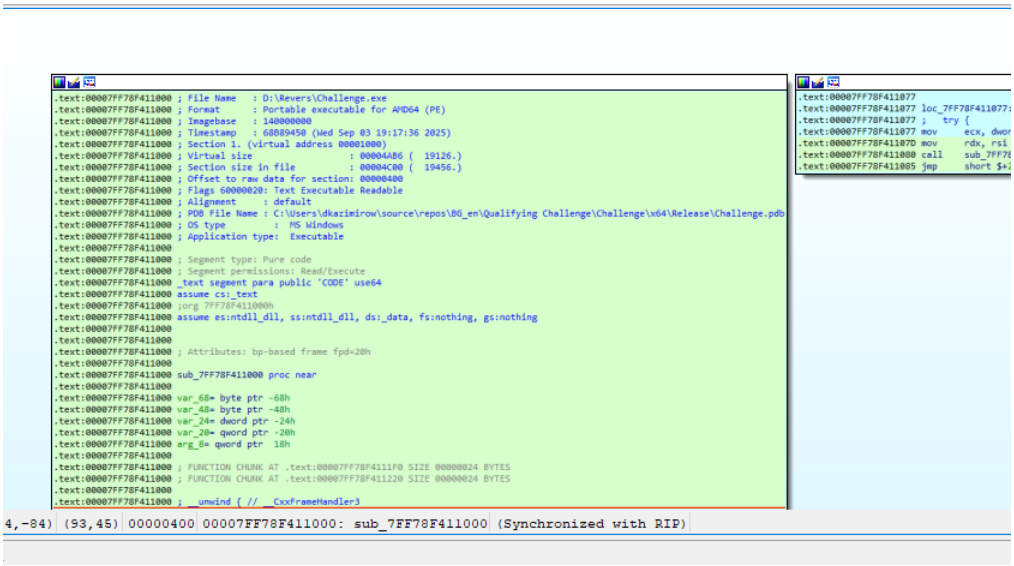
Here you give Alt + X for example to see the function of the string “ Authentication” failed !...” and gives that error.

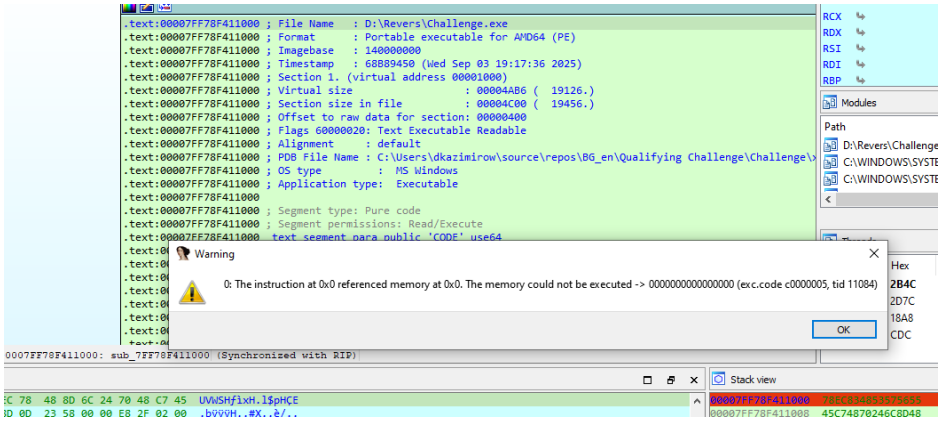
But I don't have much time and the truth is that someone with greater ability could generate a script if a pattern existed and remove the junk part and thus reconstruct the code, but that's not my case.

First I'm going to set a Hardware breakpoint – and enable “ Enable” Tracing ” -

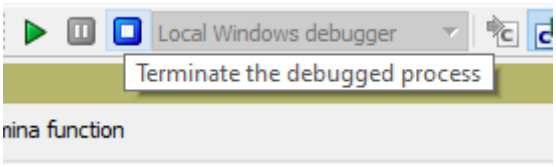
With this, I can temporarily “rebuild” it, and realize how to create the password :



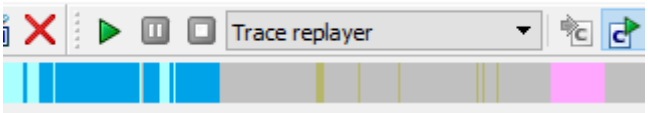




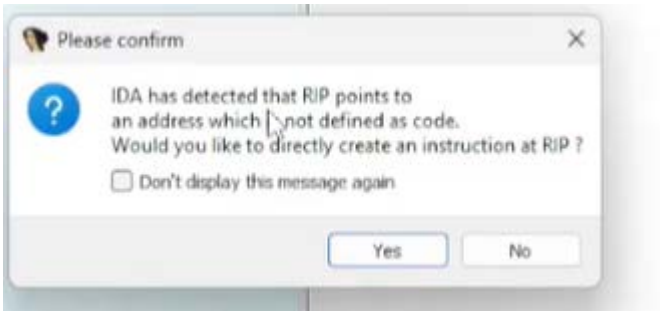
It notifies us that it has ended.



When we get to this point, we stop the Debugger and then select “Trace replayer ”.

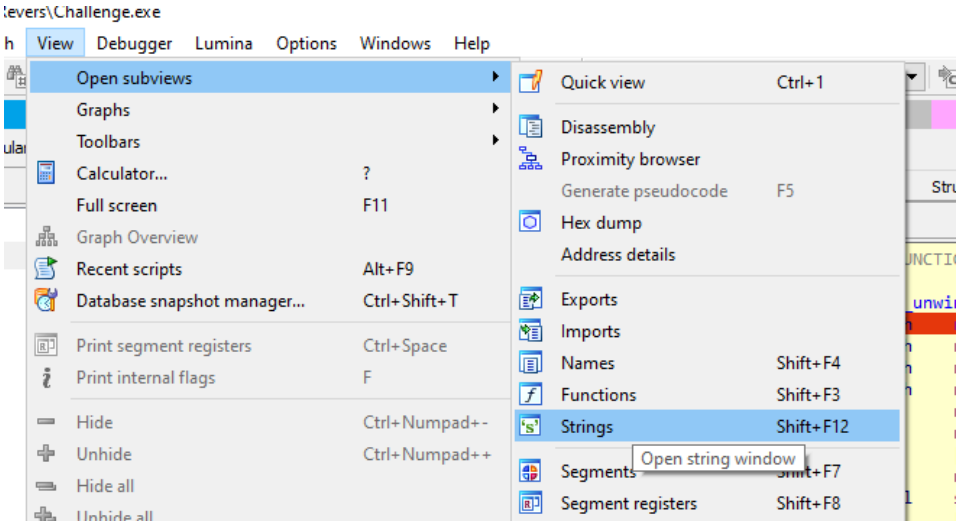


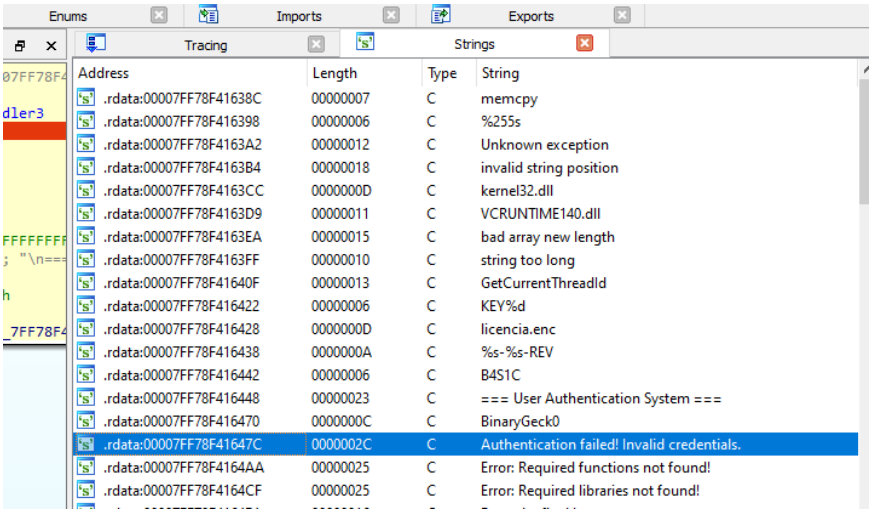
Surely when trying to view some function or something that is damaged this message will appear:

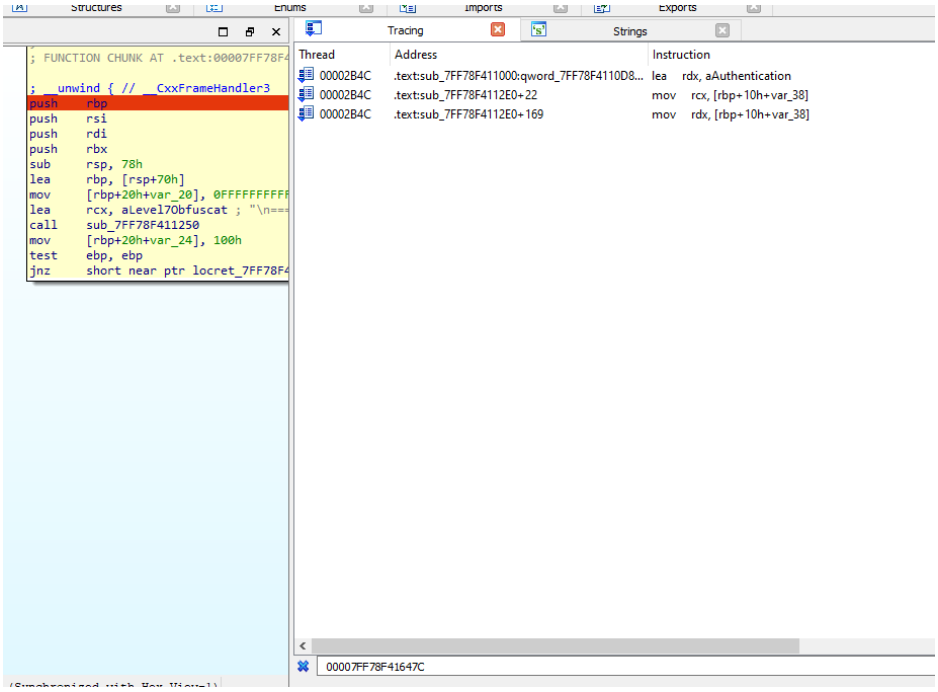


So as not to bother anyone, I checked it so it wouldn't show again and I put "Yes."

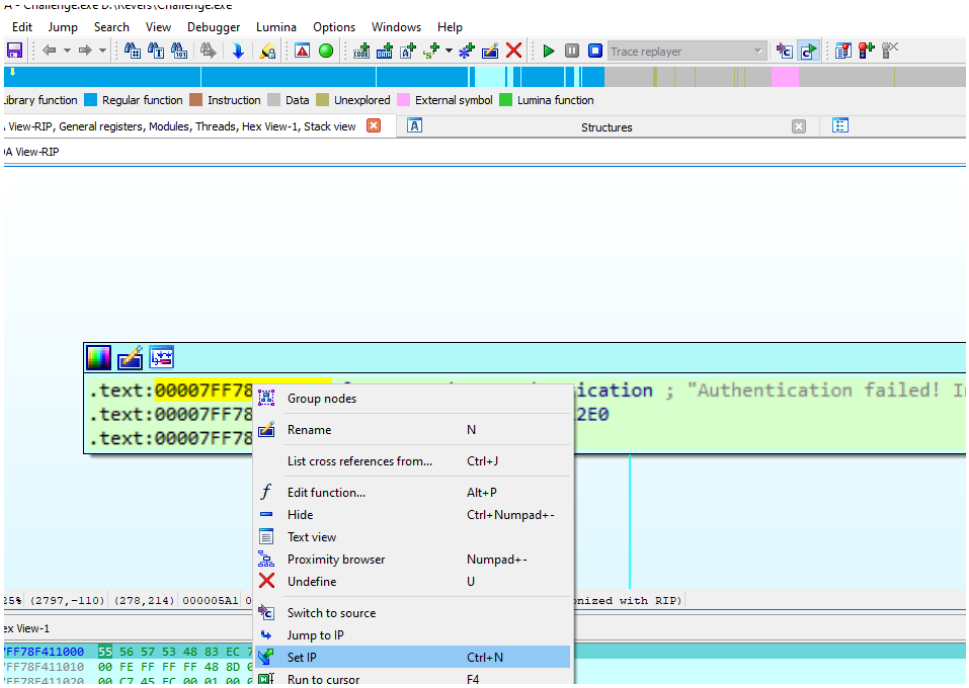
Then we look for the string :



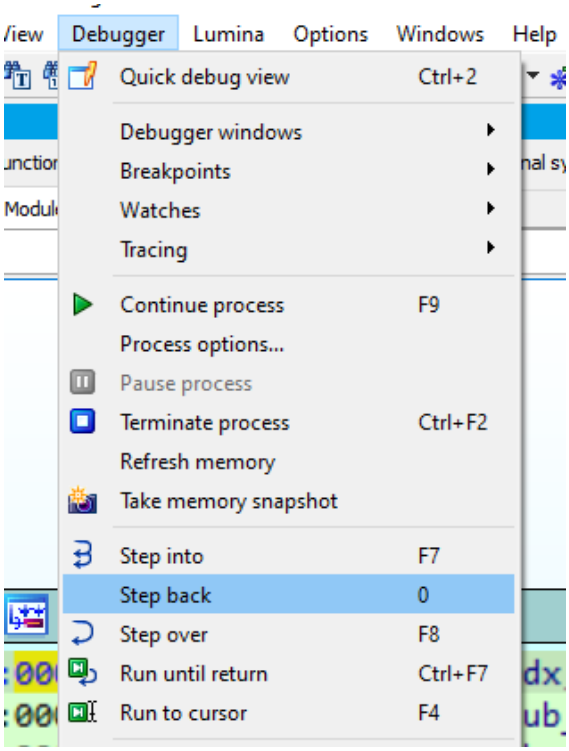




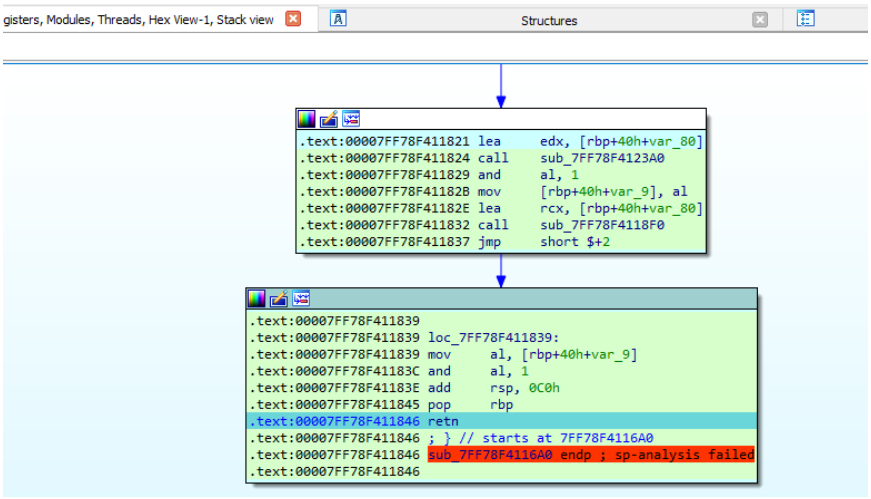
We see that its address is 00007FF6529F647C – so we look for it in the “ Tracing ” window.



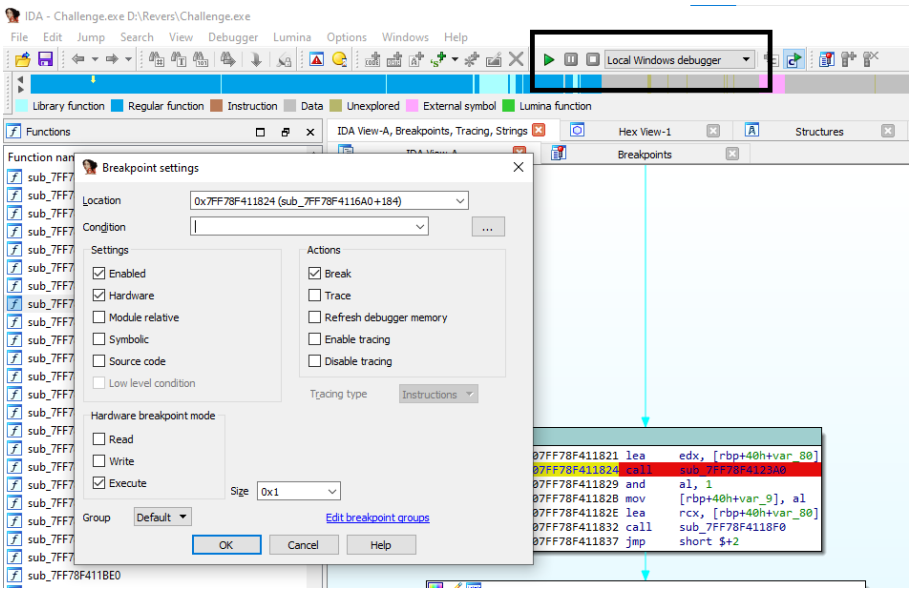
Remember to take note of the address where it is – in this case it was: 00007FF78F4111A1 (it varies) – and there we right click -> Set IP -



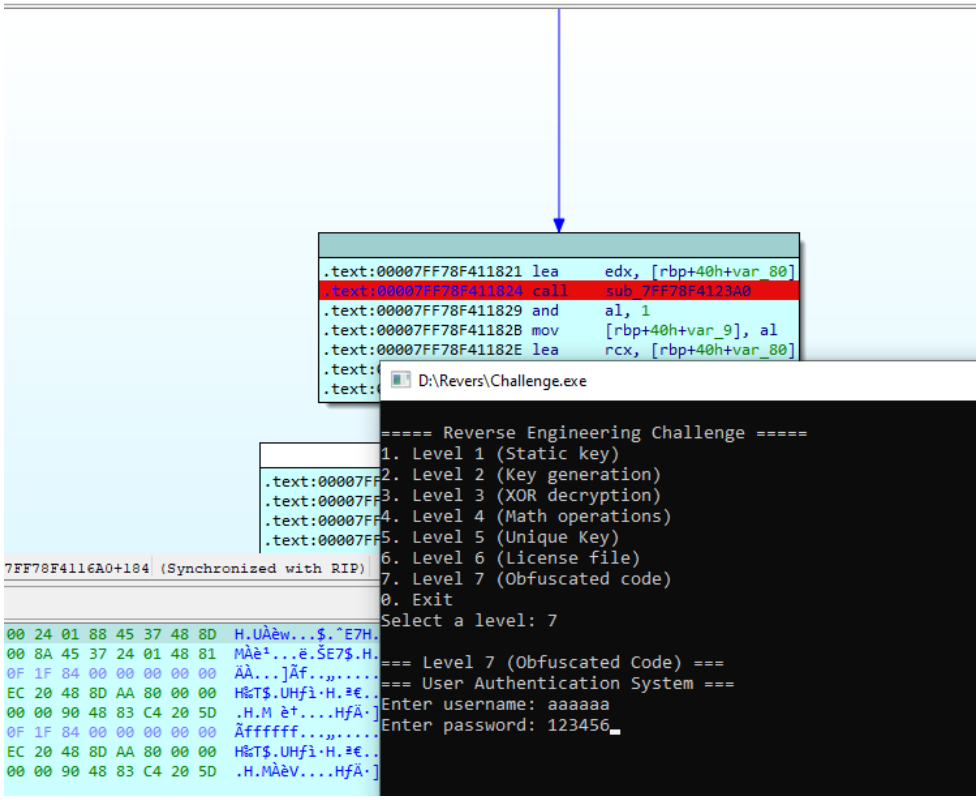
Then with Step back (in my case with number 0) we are going to reconstruct the section of the code that interests us.

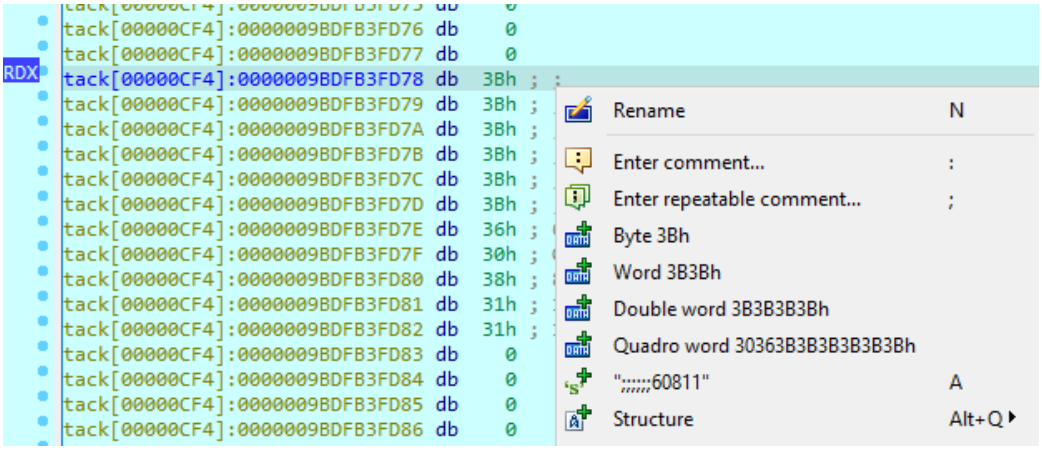


Then we set a hardware breakpoint – it is important to choose “Local Windows debugger ” again -

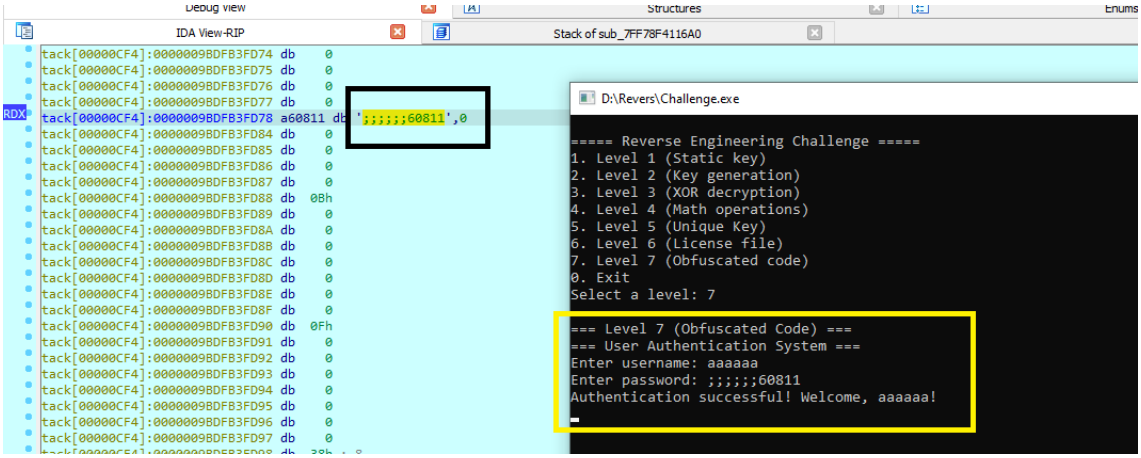


And we trace until we reach this call sub_77FF78F4123A0.



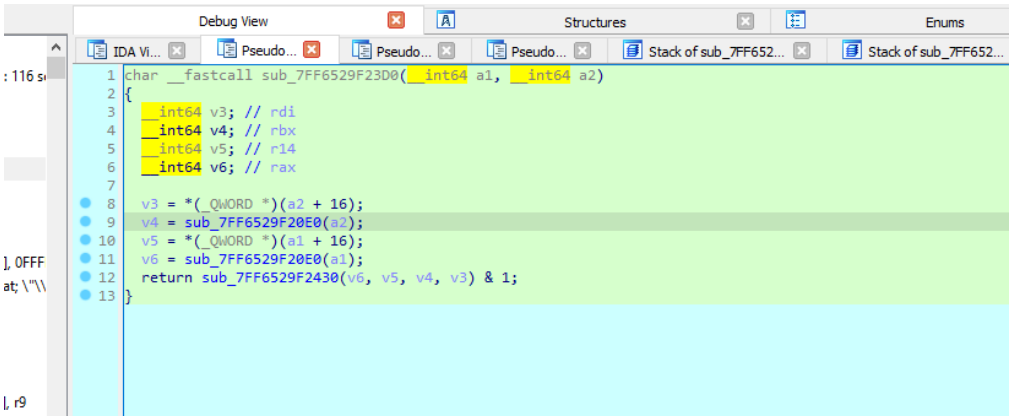


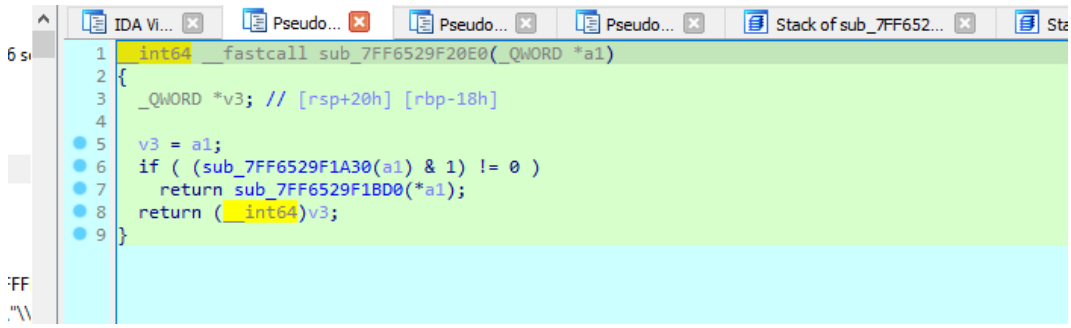
In the line that says “var_80” to see its contents, we right -click and select the corresponding option, or we type 'A' and create the string .

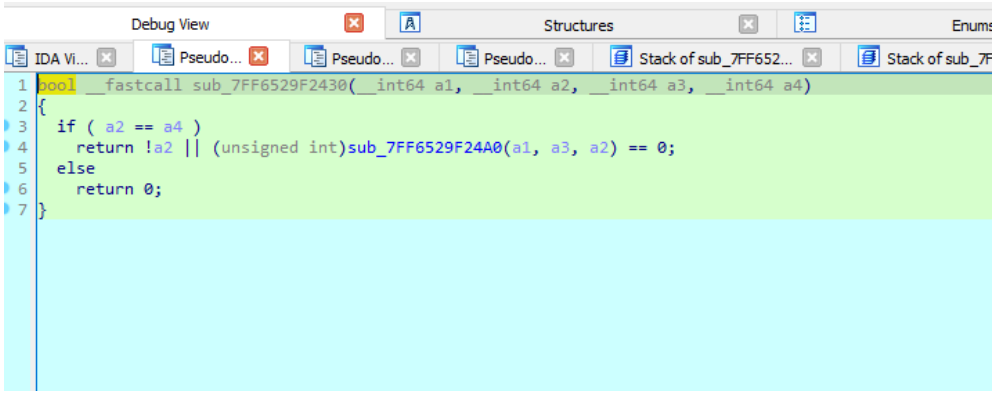


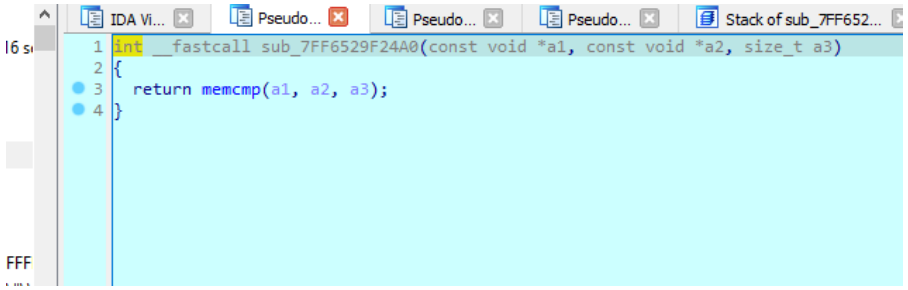
That would be the solution. Now, some personal details I'd like to share:

While I was tracing , it was actually quite complicated for me, with F5 – that is, with the pseudocode – I found some structures that caught my attention:



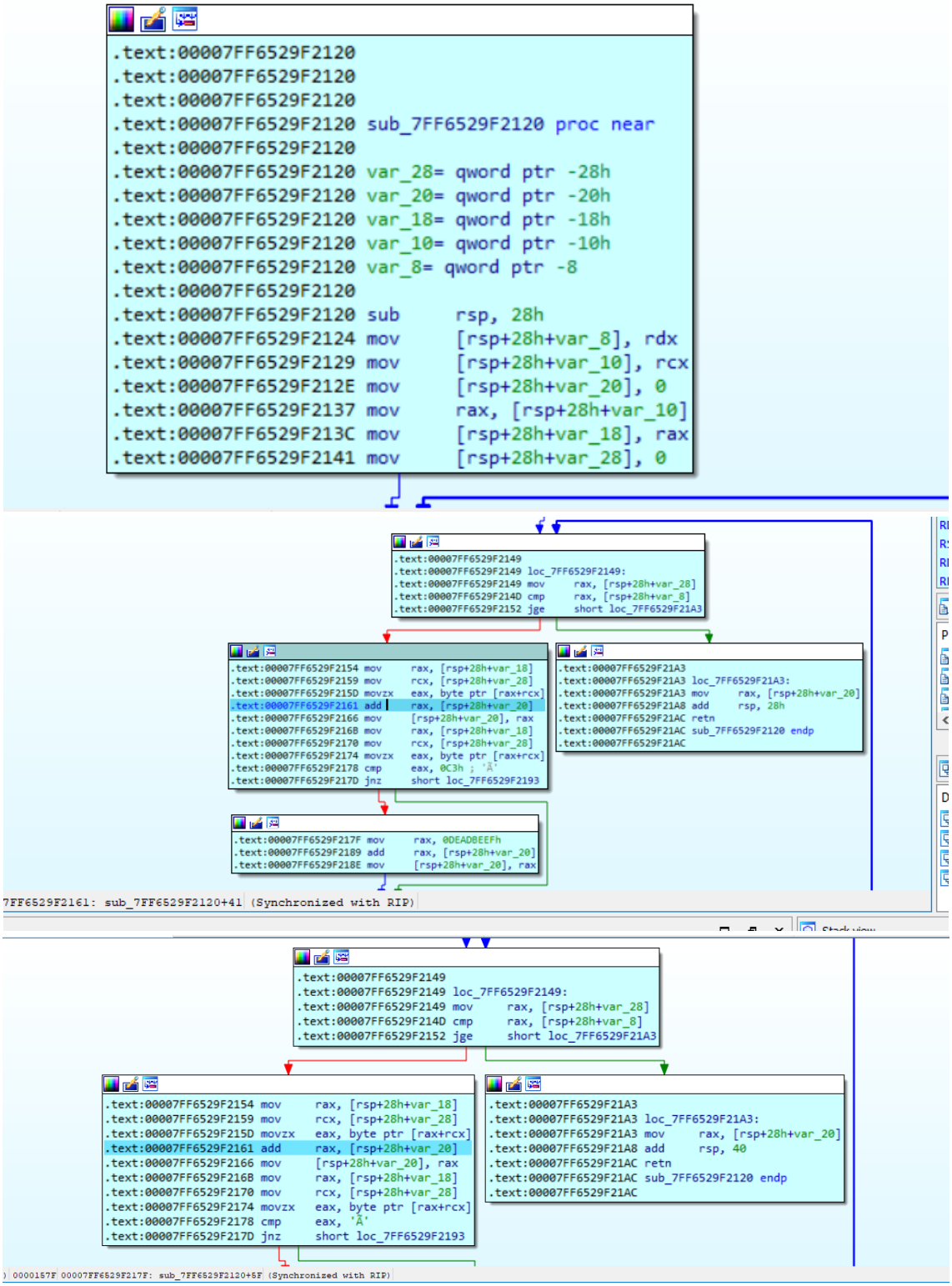


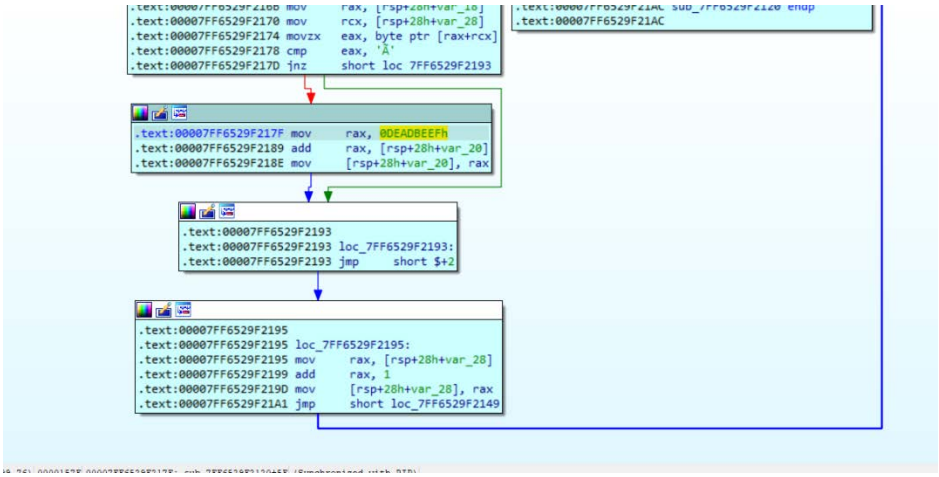




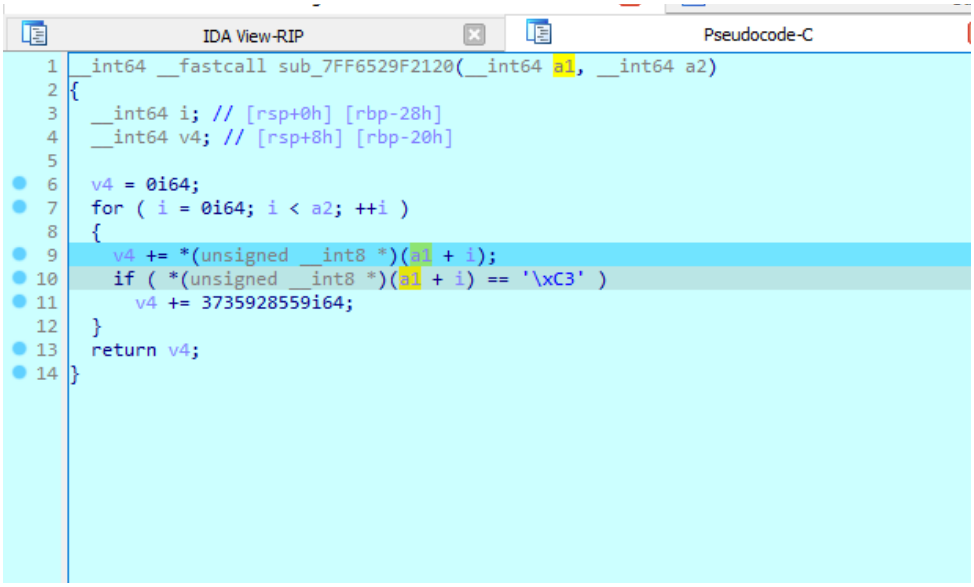
All this is just to confuse you, what matters is that it manages to perform the operation with memcmp .

Then, continuing with the tracing, I arrived at what I believe is the most important loop and where – although some details are missing – most of the calculations are performed:





Finally, I leave a screenshot of what the pseudocode would be according to IDA PRO:



Final conclusion

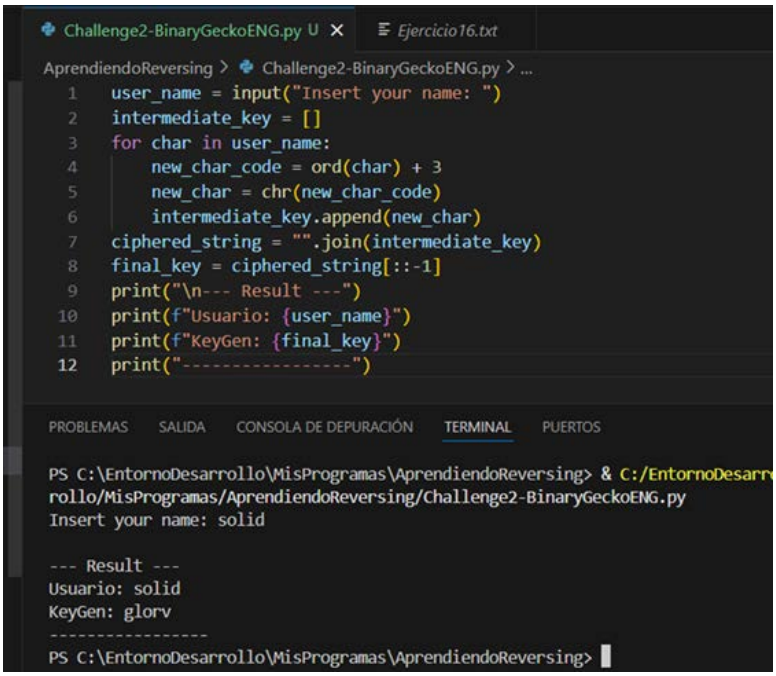
As a final note, I didn't want to miss the opportunity to thank you for the challenges. They were really fun, testing my knowledge and, at times, my patience. I think I can say that on a personal level, I feel satisfied. When I started, I thought I wouldn't even be able to solve the first one, but to my surprise, I was able to complete all of them.

Knowledge from different tools such as IDA PRO (which I had used for some things up to now but not at the level I required for this series of challenges) was applied.

I was able to gain more confidence in developing scripts (which I don't usually do for various reasons). Each of the challenges made me think differently, but it was also a reflection of the current knowledge I currently have, and that I require more knowledge to continue advancing in the world of reverse engineering.

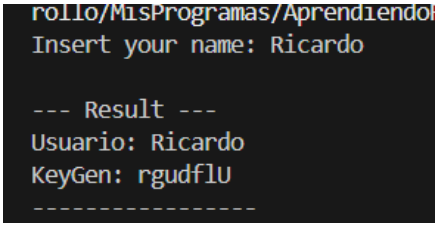
Scripts

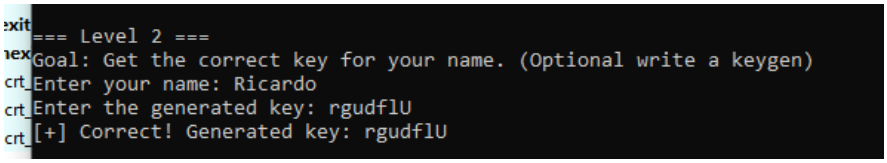
Challenge2-BinaryGeckoENG.py

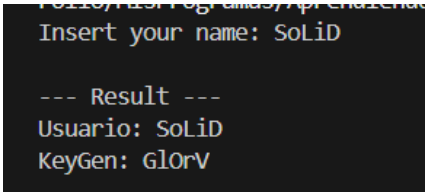


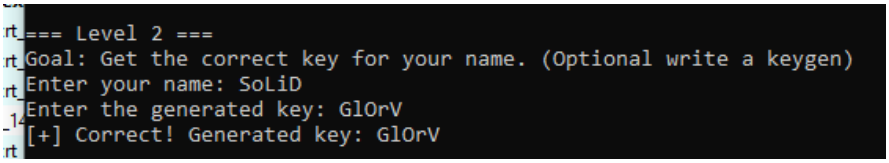
Note: This is actually the second version I made, because the first one didn't take case sensitivity into account - if the name said " Karasu " and provided the wrong key, so that was fixed:

I leave some tests carried out:



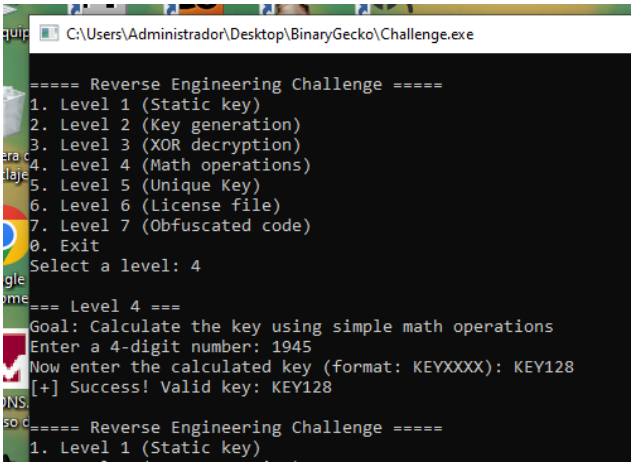




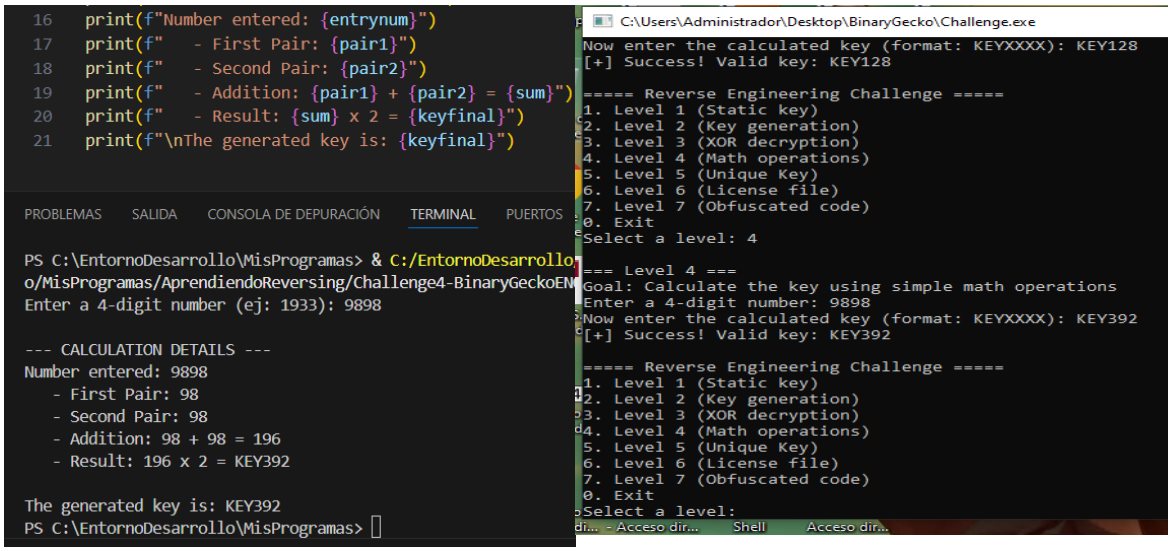


Challenge4-BinaryGeckoENG .py

```python
valid_entry = False
while not valid_entry:
    entrynum = input("Enter a 4-digit number (ej: 1933): ")
    if len(entrynum) == 4 and entrynum.isdigit():
        valid_entry = True
    else:
        print("Error: Please enter exactly 4 digits. Please try again.")
pair1_str = entrynum[0:2]
pair2_str = entrynum[2:4]
pair1 = int(pair1_str)
pair2 = int(pair2_str)
sum = pair1 + pair2
keyfinal = sum * 2
keyfinal = "KEY" + str(keyfinal)
print("\n--- CALCULATION DETAILS ---")
print(f"Number entered: {entrynum}")
print(f"    - First Pair: {pair1}")
print(f"    - Second Pair: {pair2}")
print(f"    - Addition: {pair1} + {pair2} = {sum}")
print(f"    - Result: {sum} x 2 = {keyfinal}")
print(f"\nThe generated key is: {keyfinal}")
```



Other tests performed



Challenge5-BinaryGeckoENG.py

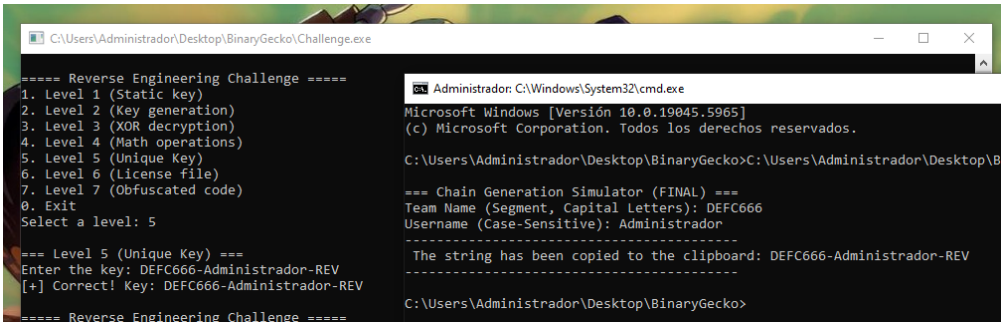
```python
import socket
import getpass
import pyperclip

namepccomplete = socket.gethostname()

if '-' in namepccomplete:
    namesegment = namepccomplete.split('-')[-1]
else:
    namesegment = namepccomplete
namesegment = namesegment.upper()
username = getpass.getuser()
basestrings = namesegment + "-" + username + "-REV"
try:
    pyperclip.copy(basestrings)
    print("\n=== Chain Generation Simulator (FINAL) ===")
    print(f"Team Name (Segment, Capital Letters): {namesegment}")
    print(f"Username (Case-Sensitive): {username}")
    print("-----------------------------------------")
    print(f" The string has been copied to the clipboard: {basestrings}")
    print("-----------------------------------------")
except pyperclip.PyperclipException as e:

    print("\nERROR: Could not access clipboard.")
    print("Make sure you have a graphical environment or the necessary dependencies (e.g. xclip/xsel on Linux).")
    print(f"The generated string is: {basestrings}")
    print(f"Error details: {e}")
```

I know that whoever reads this document knows infinitely more than I do, but just in case I clarify that for the script to work you must have pypyerclip installed – From CMD (Windows) -> pip install pyperclip (this is so that when I generate the name of the PC and concatenate it and create the corresponding key, it remains in the clipboard.



Challenge6-BinaryGeckoENG.py

```python
import os
import struct
import sys
destination_directory = r"C:\Users\Administrador\Desktop\BinaryGecko"
filename = "licencia.enc"
pathcomplete = os.path.join(destination_directory, filename)
head = struct.pack("<I", 1279869765)
substring1 = b"BinaryGeck0\x00"
stuffed = b"\x00\x00\x00\x00"
substring2 = b"9898\x00"
final = head + substring1 + stuffed + substring2

try:
    os.makedirs(destination_directory, exist_ok=True)
    with open(pathcomplete, "wb") as f:
        f.write(final)

    print(f"    Success! File '{filename}' generated.")
    print(f"   Size: {len(final)} bytes")
    print(f"   Save path: {pathcomplete}")

except PermissionError:
    print(f" Permissions error: Could not write to path '{destination_directory}'.", file=sys.stderr)
    print("   Make sure you run the script with sufficient permissions.", file=sys.stderr)
except Exception as e:
    print(f"    Unexpected error while writing file: {e}", file=sys.stderr)
```